从 C++到 AI 计算：第三册实践卷

本地量化推理引擎、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践卷只做一件事：把这些概念压到同一个可运行项目里，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 `books/cpu-volume-3/labs/linux_cpu_inference`。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest gate、int8 linear、packed/AVX2 路径、tiny reference decoder、GPT-2 reference path、KV cache trace、7B compute ledger、serving SLO probe、自研 GPT-2 smoke、真实 small 开源模型 smoke、CLI、benchmark 和 CTest。实践卷不复制这套代码，避免两份实现漂移；所有练习都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

> 第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
SLO	Service Level Objective	服务目标，例如 TTFT、TPOT、p95/p99。
TTFT	Time To First Token	从请求进入到首 token 输出的时间。
TPOT	Time Per Output Token	decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark。

本卷结构

本卷按第三册原书的四个大块组织。每一章都对应原书的一组主题文件，并把概念落到同一条实践主线：`linux_cpu_inference`。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并解释 int8 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告，并显式运行 LCQI 自研 GPT-2 safetensors smoke 和一个 small 级别开源 GGUF 模型 smoke，而不是只停留在 tiny fixture。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路与项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路与项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 2 章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第 2 章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 3 章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第 3 章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 baseline 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 shape sweep 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 4 章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第 4 章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2,-1,3,0]` 和输入 `[1,2,-1,0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 scale 作用位置、累加宽度、误差阈值和 golden 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 tokenizer、q/k/v、`kv_cache_slot`、attention 和 logits 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 activation；position、layer、head、layout 和生命周期必须进入 trace 这条规律要写进报告，不要只写 “运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为 “接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 6 章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `argmax`，核心命令或测试入口是 `lcqi_trace`。

Listing 6.1 第 6 章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 `tokenizer_contract` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 serving probe 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、cancel reclaim、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：推理服务不是只看 tokens/s；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 prompt/new token、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第 9 章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 safetensors header、metadata、tensor name、dtype、shape、`data_offsets`、payload boundary 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第 9 章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 safetensors fixture。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 offset 越界和 dtype/shape 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、shape、dtype、offset 和 payload 都必须在 kernel 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 manifest gate 报告：合法 fixture 和两个坏 fixture 的字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

本章新增一个明确边界：tiny fixture 负责手算和逐层 oracle；LCQI GPT-2 reference path 负责证明自研代码已经能加载标准 GPT-2 safetensors、执行 byte-level BPE、跑 GPT-2 forward 并 greedy 生成；SmolLM2 small smoke 负责证明外部 llama.cpp 路径能加载 GGUF small 模型。三者不能互相冒充。tiny MLP 算对，不等于 GPT-2 完成；GPT-2 reference 能出一个 token，不等于高性能 KV cache runtime 完成；SmolLM2 能输出文本，也不等于 LCQI 自研 GGUF 后端完成。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里有三组核心对象。第一组是 `tiny_tokenizer.txt`、reference decoder 和 `lcqi_trace`，用于手算 token 合同和逐 checkpoint 追踪。第二组是 `gpt2_reference.hpp/.cpp`、`lcqi_gpt2` 和 `tools/run_gpt2_smoke.py`，用于自研 GPT-2 safetensors 路径。第三组是 `tools/run_smollm2_small_smoke.py`，用于下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，再通过 llama.cpp 在 CPU 上实际生成 assistant 文本。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-smollm2-smoke
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环。这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通

GPT-2 不是 LLaMA。它使用 LayerNorm，不是 RMSNorm；使用 learned absolute position embedding，不是 RoPE；attention 里的 Q/K/V 在 HuggingFace 权重中是 packed Conv1D 权重；MLP 使用 GELU，不是 SwiGLU；`lm_head` 通常和 token embedding 共享权重。把 GPT-2 硬塞进原来的 LLaMA-like tiny decoder，会让概念混乱，所以 LCQI 新增了独立的 `gpt2_reference` 模块。

仓库内最小验收分两层。第一层不下载模型，运行 tiny GPT-2 fixture：

Listing 10.2 仓库内 tiny GPT-2 数学闭环

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
```

它固定 prompt ids 为 `123`，检查 logits、predicted token 和 greedy generated ids。对应测试还会临时写出一个 HuggingFace 风格 `model.safetensors`，再用 loader 读回来，覆盖 tensor name mapping、shape verifier、Conv1D 转置和 tied lm head。坏 shape 必须在 loader 阶段被拒绝。

第二层下载真实 GPT-2 资产：

Listing 10.3 LCQI 自研 GPT-2 smoke

```
make cpu3-gpt2-smoke
```

脚本下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors` 到用户缓存目录，不提交模型文件。最近一次报告写到：

Listing 10.4 GPT-2 smoke 报告关键字段

```
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明自研 LCQI C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。边界也要写清：当前实现为了可读和可验证，会重跑完整前缀，没有 KV cache、没有 packed weight、没有多线程，也没有把 logits 与 PyTorch/Transformers 的逐数值 golden 报告纳入仓库。因此它是正确性闭环，不是性能结论。

四、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来

本章的真实模型 smoke 固定为 `SmolLM2-135M-Instruct`，规模是 135M 参数，不是 tiny 随机夹具。脚本使用 GGUF 文件 `SmolLM2-135M-Instruct-Q4_K_M.gguf`，来源仓库是 `bartowski/SmolLM2-135M-Instruct-GGUF`，原模型组织是 `HuggingFaceTB`，license 字段为 `apache-2.0`。脚本不会把模型文件提交到仓库，而是缓存到用户目录；验收时检查文件大小 `105454432` 字节和 SHA256：

Listing 10.5 SmolLM2 small GGUF 文件身份

```
model_source_id=HuggingFaceTB/SmolLM2-135M-Instruct
model_gguf_repo_id=bartowski/SmolLM2-135M-Instruct-GGUF
model_file=SmolLM2-135M-Instruct-Q4_K_M.gguf
model_expected_bytes=105454432
model_expected_sha256=2↩
    ↪ e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

运行命令是：

Listing 10.6 真实 small 模型 CPU 冒烟

```
make cpu3-smollm2-smoke
```

这条命令会做四步。第一，下载或复用 GGUF 文件，并校验 SHA256，防止“文件名对了但内容不对”。第二，拉取固定 commit 的 `llama.cpp` 并构建 `llama-simple-chat`，避免依赖本机临时安装。第三，用 `-ngl0` 强制 CPU 路径，输入一个短 prompt，确认程序能走过 tokenizer、chat template、prefill、decode 和 sampler。第四，把报告写到：

Listing 10.7 真实模型 smoke 报告路径

```
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

报告里必须至少出现这些字段：`model_expected_sha256`、`llama_cpp_commit`、`command`、`exit_code=0`、`validation=PASS` 和 `assistant_response`。如果只有下载日志，没有 assistant response，不能算通过；如果 assistant response 内容答错了，也不能改写成“质量通过”。本章只验证真实模型资产可加载、可 tokenize、可 decode、可生成非空文本。

五、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：运行 GPT-2 smoke。报告不许只贴“生成了 John”，必须解释 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 各自对应哪段代码，为什么 GPT-2 需要 byte-level BPE，为什么 HuggingFace Conv1D 权重要转置，为什么 `lm_head` 可以 tied 到 embedding。

任务四：运行真实 small 模型 smoke。报告不许只贴“命令执行成功”，必须解释脚本为什么校验模型 hash、为什么固定 `llama.cpp` commit、为什么使用 `-ngl0`，以及为什么 assistant response 非空只是 smoke，不是质量评测。

任务五：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。GPT-2 反例可以是：tensor name 缺失、shape 和 config 不一致、vocab 中缺少 BPE merge 后 token、`max_new_tokens` 超过 context。真实模型 smoke 的反例可以是：模型 SHA256 不匹配、网络不可用、`cmake` 缺失、assistant response 为空、命令超时。每个反例都要写清期望处理方式。

六、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。新增 GPT-2 smoke 后，最低验收还包括：真实 GPT-2 文件身份有 hash，prompt ids 可解释，generated ids 可解释，文本反解码可解释，并明确声明这不是高性能推理结论。新增 SmolLM2 smoke 后，最低验收还包括：模型身份有 hash，模型规模不是 tiny，CPU 路径实际运行，报告里有非空 assistant response，并明确声明这不是自研 LCQI 完整 GGUF 支持。

递进大作业：提交 GPT-2 reference path 的下一阶段迁移清单：KV cache、prefill/decode 分离、packed weight、线程并行、Transformers golden logits、更多模型方言 name mapping、分片 safetensors 和性能报告。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 12 章实践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `avx2`，核心命令或测试入口是 `lcqi_bench`。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / avx2
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 `average_us` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `StepTrace`，核心命令或测试入口是 `lcqi_trace`。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 `copy_time_ns` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 trace、memory、backend 或 serving 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、trace 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 reference、unit test、trace golden、benchmark gate、SLO gate、coverage、CI 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 test 保护哪条合同。如果输出里没有 `100% tests passed` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、trace 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：README、命令记录、报告索引、测试结果、benchmark 表、trace 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

完整源码清单

本附录从 books/cpu-volume-3/labs/linux_cpu_inference 直接读入贯穿项目源码。实践卷正文只解释每章要看哪一段；真正的闭环必须回到完整工程：头文件、实现、CLI、benchmark、trace、测试和模型 fixture 都要能一起构建。

一、构建入口和项目说明

Listing 16.2 第三册 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)

if(CPU_VOLUME_3_BUILD_LABS)
    enable_testing()
    add_subdirectory(labs/linux_cpu_inference)
endif()
```

Listing 16.3 linux_cpu_inference CMakeLists.txt

```
add_library(lcqi STATIC
    src/gpt2_reference.cpp
    src/inference.cpp
    src/int8_kernels.cpp
    src/model.cpp
    src/reference_decoder.cpp
    src/safetensors.cpp
    src/tokenizer.cpp
)

if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
        set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2;-mfma")
        target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
    endif()
endif()

if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
    message(WARNING "LCQI is written for the book's local Linux CPU target; ↩
    ↪ current system is ${CMAKE_SYSTEM_NAME}.")
```

```
endif()

target_include_directories(lcqi
    PUBLIC
        ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(lcqi PUBLIC cxx_std_20)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_cli
    src/main.cpp
)

target_link_libraries(lcqi_cli PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_bench
    src/bench.cpp
)

target_link_libraries(lcqi_bench PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_trace
    src/trace.cpp
)

target_link_libraries(lcqi_trace PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_gpt2
    src/gpt2_cli.cpp
)

target_link_libraries(lcqi_gpt2 PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
endif()
```

```
73
74 add_executable(lcqi_ledger
75     src/ledger.cpp
76 )
77
78 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
79     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
80 endif()
81
82 add_executable(lcqi_serving_probe
83     src/serving_probe.cpp
84 )
85
86 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
87     target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
88 endif()
89
90 find_package(dnnl QUIET)
91 if(dnnl_FOUND)
92     add_executable(lcqi_onednn_bench
93         src/onednn_bench.cpp
94     )
95     target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
96     target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
97     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
98         target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -↩
    ↪ Wpedantic)
99     endif()
100 endif()
101
102 add_executable(lcqi_tests
103     tests/lcqi_tests.cpp
104 )
105
106 target_link_libraries(lcqi_tests PRIVATE lcqi)
107
108 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
109     target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
110 endif()
111
112 add_test(NAME lcqi_tests COMMAND $<TARGET_FILE:lcqi_tests>)
113 add_test(NAME lcqi_gpt2_tiny COMMAND $<TARGET_FILE:lcqi_gpt2>)
114 set_tests_properties(lcqi_gpt2_tiny PROPERTIES
115     PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
116 )
117 add_test(NAME lcqi_trace COMMAND $<TARGET_FILE:lcqi_trace>)
118 set_tests_properties(lcqi_trace PROPERTIES
119     PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
120 )
121 add_test(NAME lcqi_ledger COMMAND $<TARGET_FILE:lcqi_ledger>)
122 set_tests_properties(lcqi_ledger PROPERTIES
123     PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
```

```
)
add_test(NAME lcqi_serving_probe COMMAND $<TARGET_FILE:lcqi_serving_probe>)
set_tests_properties(lcqi_serving_probe PROPERTIES
    PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
)
```

Listing 16.4 linux_cpu_inference README.md

```
# Linux CPU Quantized Inference

这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache、CPU/GPU 后端、正确性报告和性能对标。

目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变当前切片的语义合同。

当前版本支持：

- 教学文本模型格式 `LCQI_MODEL_V1`
- 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
- safetensors header manifest 解析：metadata、tensor name、dtype、shape、data offsets、offset 边界和 dtype/shape 字节数校验
- 两层 MLP 教学切片
- int8 权重加 per-layer scale
- float 输入和激活
- 量化线性层、ReLU、分类 argmax
- int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
- tiny reference decoder：RMSNorm、float linear、RoPE、GQA KV cache、decode attention、SwiGLU、lm head
- GPT-2 reference path：byte-level BPE tokenizer、`config.json`、F32/F16/BF16 safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation、KV cache greedy generation 和分阶段 benchmark
- reference trace CSV：从 prompt/tokenizer 合同到 logits/sampler/token，逐 checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values 摘要
- 7B ledger CSV：输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限
- serving SLO probe CSV：输出 admission、batch state、KV 预算、queue wait、TTFT、TPOT、取消回收和拒绝率
- 自研 GPT-2 冒烟：加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
- 自研 GPT-2 benchmark 对比：同一个 GPT-2 F32 safetensors 模型分别跑 full-prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎参考
- 真实 small 开源模型冒烟：通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量
```

```
    ↪ 化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和↩
    ↪ 性能行写入报告
- CLI 推理和简单 benchmark
- CTest 正确性测试

后续扩展方向：

- 真实 7B 级模型 metadata、tokenizer 和量化权重导入
- SentencePiece tokenizer、GGUF 权重导入、safetensors 分片加载和更多模型方言 ↩
    ↪ name mapping
- GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
- KV cache 分页、内存规划和可选量化
- CPU packed weight、SIMD、线程和 NUMA 优化
- GPU backend adapter 和 device kernel
- 与现有框架的 prefill/decode/内存/误差对标报告

构建：

```bash
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
```

运行：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
  books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
  1.0 2.0 -1.0 0.5 1000
```

kernel benchmark：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
```

输出 CSV 字段：

```text
target_arch,input_size,output_size,output_block,repeat,backend,available,↩
    ↪ average_us,max_abs_diff,checksum
```

当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 ↩
    ↪ 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `↩
    ↪ available=0`，避免把源码存在误报成当前环境实测性能。

这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↩
    ↪ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继↩
```

续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对照、sanitizer、fuzz 和 CI。

safetensors manifest gate:

`lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant descriptor 和 first token trace。

reference decoder trace:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
```

输出 CSV 字段:

```text
name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
```

这条 trace 固定 prompt fixture `tok1 tok2 tok3`，tokenizer 输出完整 `tokens=[0,1,2,3,4]`，其中 `0/4` 是 BOS/EOS；decoder trace 会剥离 BOS/EOS 后进入 tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同，避免把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`lcqi_tests` 会检查 tokenizer contract **hash**、关键 checkpoint 的 shape、stride、dtype、layout 和 logits golden，CTest 也会直接运行 `lcqi_trace`。

当前 tokenizer fixture 的关键输出:

```text
tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-chat-template-v1
```

GPT-2 自研 reference smoke:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
```

第一条命令不需要下载模型，直接运行仓库内 tiny GPT-2 fixture，输出固定 prompt ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖

tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16 safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape 拒绝。

第二条命令显式依赖网络，只下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 到 `~/.cache/lcqi-gpt2-smoke/openai-community--gpt2`，模型文件不进入 Git。脚本会校验四个文件的 bytes 和 SHA256，构建 `lcqi_gpt2`，运行：

```text
Hello, my name is
```

当前验证报告写到：

```text
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
```

最近一次本机报告的关键行是：

```text
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：

```bash
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
  --benchmark --engine cached \
  ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
  "Hello, my name is" 4
```

`make cpu3-gpt2-benchmark-compare` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
```

最近一次本机 aarch64/Qualcomm 报告中，LCQI full-prefix 在 GPT-2 F32 上生成 4 个 token 的生成阶段为 `2522.81 ms`，约 `1.586 token/s`；LCQI cached-KV 同样输出 `Hello, my name is John. I'm`，生成阶段为 `826.152 ms`，约 `4.842 token/s`。cached-KV 的第一个新 token 来自 prefill 最后一步，后续 decode 实际执行 3 个 forward step，耗时 `309.524 ms`，约 `9.692 step/s`。同机 llama.cpp 使用 SmolLM2-135M-Instruct Q4_K_M 的 `llama-bench -p 5 -n 4` 作为成熟引擎参照，prefill `76.41 token/s`、decode `27.83 token/s`。这个对照

```
    ↪ 不是同模型质量排名：LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的↩
    ↪ 是 GGUF 量化 small instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每↩
    ↪ 步重算前缀”的主要算法错误，但还缺 packed weight、量化 matmul、SIMD/SVE/↩
    ↪ NEON kernel、线程并行、mmap/lazy load、采样器和成熟的内存调度。

它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/↩
    ↪ PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache↩
    ↪ 、量化权重执行和高性能多线程 kernel。

7B ledger:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
  > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
```

输出 CSV 字段：

```text
section,item,value,unit,notes
```

这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode↩
    ↪ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s ↩
    ↪ 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公↩
    ↪ 式漂移。

serving SLO probe:

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe↩
    ↪  \
  > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

输出 CSV 字段：

```text
row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,↩
    ↪ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,↩
    ↪ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,↩
    ↪ metric_value
```

probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按↩
    ↪ `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回↩
    ↪ `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`↩
    ↪ rejection_rate` 等服务化指标。

真实 small 开源模型 smoke:

```bash
```
```

````
make cpu3-smollm2-smoke
```

这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型文件缓存到 `~/.cache/lcqi-smollm2-small-smoke`，模型不进入 Git。固定模型是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：

```text
2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短 prompt，检查 assistant response 非空，并生成：

```text
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

这份报告只能证明真实 GGUF small 模型已经完成加载、tokenizer/chat template、prefill、decode 和文本输出。它不能证明 LCQI 自研 C++ 引擎已经支持完整 GGUF、不能证明回答质量正确，也不能替代 tiny fixture 的逐层 golden trace。tiny fixture 继续负责可手算语义和回归定位；SmolLM2 smoke 负责把“真实开源模型能在本机跑起来”变成可复查证据。
````

Listing 16.5 tools/run_smollm2_small_smoke.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import re
import shlex
import shutil
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path


MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
MODEL_URL = (
    "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
    f"/resolve/main/{MODEL_FILE_NAME}"
)
MODEL_EXPECTED_BYTES = 105454432
```
```

```
MODEL_EXPECTED_SHA256 = (
    "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
)
LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
LLAMA_TARGET = "llama-simple-chat"
DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
DEFAULT_CONTEXT_TOKENS = 512
DEFAULT_TIMEOUT_SECONDS = 180
DEFAULT_BUILD_JOBS = 2


@dataclass(frozen=True)
class CommandResult:
    args: list[str]
    returncode: int
    stdout: str
    stderr: str


def script_path() -> Path:
    return Path(__file__).resolve()


def volume_dir() -> Path:
    return script_path().parents[1]


def repo_dir() -> Path:
    return script_path().parents[3]


def default_cache_dir() -> Path:
    explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
    if explicit_cache:
        return Path(explicit_cache).expanduser()
    xdg_cache = os.environ.get("XDG_CACHE_HOME")
    cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "
    ↪ .cache"
    return cache_root / "lcqi-smollm2-small-smoke"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-smollm2-small-model-smoke.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def require_tool(name: str) -> str:
    path = shutil.which(name)
```

```
    if path is None:
        raise RuntimeError(f"required tool not found in PATH: {name}")
    return path


def run_command(
    args: list[str],
    *,
    cwd: Path | None = None,
    input_text: str | None = None,
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
            input=input_text,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=stderr)

    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]

```

```python

def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def verify_model_file(path: Path) -> tuple[bool, str, int]:
    if not path.exists():
        return False, "", 0
    actual_bytes = path.stat().st_size
    actual_sha = sha256_file(path)
    return (
        actual_bytes == MODEL_EXPECTED_BYTES
        and actual_sha == MODEL_EXPECTED_SHA256,
        actual_sha,
        actual_bytes,
    )


def download_model(model_path: Path, *, force_download: bool, no_download: bool) -> None:
    model_path.parent.mkdir(parents=True, exist_ok=True)

    if force_download and model_path.exists():
        model_path.unlink()

    is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
    if is_valid:
        print(f"[lcqi-smoke] model cache hit: {model_path}")
        return

    if model_path.exists():
        print(
            "[lcqi-smoke] cached model hash mismatch, redownloading: "
            f"bytes={actual_bytes}, sha256={actual_sha}"
        )
        model_path.unlink()

    if no_download:
        raise RuntimeError(
            f"model file is missing or invalid and --no-download was set: {model_path}"
        )

    require_tool("curl")
    part_path = model_path.with_suffix(model_path.suffix + ".part")
    if part_path.exists():
        part_path.unlink()

```

```
    print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{MODEL_FILE_NAME}")
    result = run_command(
        [
            "curl",
            "-L",
            "--fail",
            "--retry",
            "3",
            "-o",
            str(part_path),
            MODEL_URL,
        ]
    )
    ensure_success(result, "download SmolLM2 GGUF")

    downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(part_path)
    if not downloaded_ok:
        raise RuntimeError(
            "downloaded model did not match expected identity: "
            f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
        )
    part_path.replace(model_path)
    print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")


def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) -> Path:
    cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
    source_dir = cache_dir / "llama.cpp"
    build_dir = source_dir / "build"
    if cached_binary.exists() and os.access(cached_binary, os.X_OK) and is_pinned_llama_checkout(source_dir):
        print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
        return cached_binary

    if skip_build:
        raise RuntimeError(f"llama.cpp binary is missing and --skip-build was set: {cached_binary}")

    require_tool("git")
    require_tool("cmake")

    cache_dir.mkdir(parents=True, exist_ok=True)

    if not (source_dir / ".git").exists():
        print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
        result = run_command(
            [
                "git",
                "clone",
```

```
                "--filter=blob:none",
                "--no-checkout",
                LLAMA_CPP_REPO_URL,
                str(source_dir),
            ]
        )
        ensure_success(result, "clone llama.cpp")

    print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {LLAMA_CPP_COMMIT}")
    fetch = run_command(
        ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", LLAMA_CPP_COMMIT]
    )
    ensure_success(fetch, "fetch pinned llama.cpp commit")
    checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach", LLAMA_CPP_COMMIT])
    ensure_success(checkout, "checkout pinned llama.cpp commit")

    print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
    configure = run_command(
        [
            "cmake",
            "-S",
            str(source_dir),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
            "-DLLAMA_BUILD_TESTS=OFF",
            "-DLLAMA_BUILD_EXAMPLES=ON",
            "-DLLAMA_BUILD_SERVER=OFF",
            "-DGGML_NATIVE=OFF",
        ]
    )
    ensure_success(configure, "configure llama.cpp")

    print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
    build = run_command(
        ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", str(jobs)]
    )
    ensure_success(build, f"build {LLAMA_TARGET}")

    if not cached_binary.exists():
        raise RuntimeError(f"expected binary was not produced: {cached_binary}")
    return cached_binary


def is_pinned_llama_checkout(source_dir: Path) -> bool:
    if not (source_dir / ".git").exists():
        return False
```

```
    result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
    if result.returncode != 0:
        return False
    return result.stdout.strip() == LLAMA_CPP_COMMIT


def strip_ansi(text: str) -> str:
    return re.sub(r"\x1b\[[0-?]*[ -/]*[@-~]", "", text)


def extract_response(stdout: str) -> str:
    cleaned = strip_ansi(stdout).replace("\r", "")
    chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if chunk.strip()]
    if not chunks:
        return ""
    return chunks[0]


def run_model_smoke(
    llama_binary: Path,
    model_path: Path,
    *,
    prompt: str,
    context_tokens: int,
    timeout_seconds: int,
) -> tuple[CommandResult, str]:
    command = [
        str(llama_binary),
        "-m",
        str(model_path),
        "-ngl",
        "0",
        "-c",
        str(context_tokens),
    ]
    print("[lcqi-smoke] running real small model generation")
    result = run_command(
        command,
        input_text=prompt.rstrip("\n") + "\n\n",
        timeout_seconds=timeout_seconds,
    )
    ensure_success(result, "run small model smoke")

    response = extract_response(result.stdout)
    if not response:
        raise RuntimeError(
            "small model smoke exited successfully but produced no assistant text\n"
            f"stdout:\n{result.stdout}\n"
            f"stderr:\n{result.stderr}"
        )
```

```
    return result, response


def current_repo_commit() -> str:
    result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()


def current_uname() -> str:
    result = run_command(["uname", "-a"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    llama_binary: Path,
    model_path: Path,
    prompt: str,
    result: CommandResult,
    response: str,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    clean_stdout = strip_ansi(result.stdout).strip()
    report = "\n".join(
        [
            "LCQI SmolLM2 Small Model Smoke",
            "",
            f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
            f"repo_commit={current_repo_commit()}",
            f"host={current_uname()}",
            f"python={sys.version.split()[0]}",
            f"cache_dir={cache_dir}",
            "",
            f"model_source_id={MODEL_SOURCE_ID}",
            f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
            f"model_file={MODEL_FILE_NAME}",
            f"model_url={MODEL_URL}",
            f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
            f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
            f"model_path={model_path}",
            "",
            f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
            f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
            f"llama_target={LLAMA_TARGET}",
            f"llama_binary={llama_binary}",
```

```
            "",
            f"prompt={prompt}",
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "",
            "validation=PASS: model hash matched, llama-simple-chat exited 0, "
            "assistant response was non-empty",
            "",
            "assistant_response:",
            response,
            "",
            "stdout_clean_tail:",
            tail(clean_stdout, 2000),
            "",
            "stderr_tail:",
            tail(result.stderr.strip(), 2000),
            "",
        ]
    )
    report_path.write_text(report, encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description=(
            "Run a real local small open-source model smoke for CPU Volume 3. "
            "The script caches llama.cpp and the GGUF model outside the Git repository."
        )
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--model-path", type=Path, default=None)
    parser.add_argument("--llama-bin", type=Path, default=None)
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
    parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--force-download", action="store_true")
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-build", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    cache_dir = args.cache_dir.expanduser().resolve()
    model_path = (
        args.model_path.expanduser().resolve()
        if args.model_path
        else cache_dir / "models" / MODEL_FILE_NAME
    )
```

```
423     report_path = args.report.expanduser().resolve()
424
425     try:
426         download_model(
427             model_path,
428             force_download=args.force_download,
429             no_download=args.no_download,
430         )
431         if args.llama_bin:
432             llama_binary = args.llama_bin.expanduser().resolve()
433             if not llama_binary.exists() or not os.access(llama_binary, os.X_OK↩
    ↪ ):
434                 raise RuntimeError(f"--llama-bin is not executable: {↩
    ↪ llama_binary}")
435         else:
436             llama_binary = ensure_llama_simple_chat(
437                 cache_dir,
438                 jobs=args.jobs,
439                 skip_build=args.skip_build,
440             )
441
442         result, response = run_model_smoke(
443             llama_binary,
444             model_path,
445             prompt=args.prompt,
446             context_tokens=args.ctx,
447             timeout_seconds=args.timeout,
448         )
449         write_report(
450             report_path,
451             cache_dir=cache_dir,
452             llama_binary=llama_binary,
453             model_path=model_path,
454             prompt=args.prompt,
455             result=result,
456             response=response,
457         )
458     except RuntimeError as exc:
459         print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
460         return 1
461
462     print(f"report={report_path}")
463     print(f"model_sha256={MODEL_EXPECTED_SHA256}")
464     print(f"assistant_response={response}")
465     return 0
466
467
468 if __name__ == "__main__":
469     raise SystemExit(main())
```

Listing 16.6 tools/run_gpt2_smoke.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import shlex
import subprocess
import sys
import urllib.request
from dataclasses import dataclass
from pathlib import Path


MODEL_REPO_ID = "openai-community/gpt2"
MODEL_REVISION = "main"
MODEL_FILE_IDENTITIES = {
    "config.json": (
        665,
        "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
    ),
    "vocab.json": (
        1042301,
        "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
    ),
    "merges.txt": (
        456318,
        "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
    ),
    "model.safetensors": (
        548105171,
        "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
    ),
}
DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 1
DEFAULT_TIMEOUT_SECONDS = 300
DEFAULT_BUILD_JOBS = 2


@dataclass(frozen=True)
class CommandResult:
    args: list[str]
    returncode: int
    stdout: str
    stderr: str


def script_path() -> Path:
    return Path(__file__).resolve()

```

```
53
54 def volume_dir() -> Path:
55     return script_path().parents[1]
56
57
58 def repo_dir() -> Path:
59     return script_path().parents[3]
60
61
62 def default_cache_dir() -> Path:
63     explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
64     if explicit_cache:
65         return Path(explicit_cache).expanduser()
66     xdg_cache = os.environ.get("XDG_CACHE_HOME")
67     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "
       ↪ .cache"
68     return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")
69
70
71 def default_build_dir() -> Path:
72     return volume_dir() / "build" / "lcqi-release"
73
74
75 def default_report_path() -> Path:
76     return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"
77
78
79 def command_line(args: list[str]) -> str:
80     return " ".join(shlex.quote(arg) for arg in args)
81
82
83 def run_command(
84     args: list[str],
85     *,
86     cwd: Path | None = None,
87     timeout_seconds: int | None = None,
88 ) -> CommandResult:
89     try:
90         completed = subprocess.run(
91             args,
92             cwd=str(cwd) if cwd else None,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=
       ↪ stderr)
102     return CommandResult(
```

```
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def model_url(file_name: str) -> str:
    return (
        f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
        f"{MODEL_REVISION}/{file_name}"
    )


def download_file(url: str, target: Path, timeout_seconds: int) -> None:
    part_path = target.with_suffix(target.suffix + ".part")
    if part_path.exists():
        part_path.unlink()
    request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-↩
    ↪ smoke"})
    with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
        with part_path.open("wb") as output:
            while True:
                block = response.read(1024 * 1024)
                if not block:
                    break
                output.write(block)
```

```
    part_path.replace(target)


def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str) -> bool:
    return (
        path.exists()
        and path.stat().st_size == expected_bytes
        and sha256_file(path) == expected_sha256
    )


def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: int) -> None:
    cache_dir.mkdir(parents=True, exist_ok=True)
    missing = [
        name
        for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items()
        if not file_identity_matches(cache_dir / name, expected_bytes, expected_sha256)
    ]
    if not missing:
        print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
        return
    if no_download:
        raise RuntimeError(
            f"missing GPT-2 model files and --no-download was set: {missing}"
        )
    for file_name in missing:
        target = cache_dir / file_name
        if target.exists():
            target.unlink()
        print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
        download_file(model_url(file_name), target, timeout_seconds)
        expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
        if not file_identity_matches(target, expected_bytes, expected_sha256):
            raise RuntimeError(
                f"downloaded GPT-2 file identity mismatch: {file_name}"
            )


def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(volume_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
```

```
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(configure, "configure LCQI")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(build, "build lcqi_gpt2")
    binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    binary: Path,
    prompt: str,
    max_new_tokens: int,
    engine: str,
    result: CommandResult,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"model_repo_id={MODEL_REPO_ID}",
        f"model_revision={MODEL_REVISION}",
        f"model_cache_dir={cache_dir}",
    ]
    for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
        path = cache_dir / file_name
        lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
        lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
        lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
        lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
    lines.extend(
        [
            f"binary={binary}",
            f"prompt={prompt}",
            f"max_new_tokens={max_new_tokens}",
            f"engine={engine}",
```

```
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "stdout_begin",
            result.stdout.rstrip(),
            "stdout_end",
            "stderr_begin",
            tail(result.stderr).rstrip(),
            "stderr_end",
        ]
    )
    passed = result.returncode == 0 and "generated_text " in result.stdout
    lines.append(f"validation={'PASS' if passed else 'FAIL'}")
    report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run LCQI self-owned GPT-2 smoke on a HuggingFace safetensors directory."
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--engine", choices=("cached", "full"), default="cached")
    parser.add_argument("--benchmark", action="store_true")
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    try:
        if args.max_new_tokens < 0:
            raise RuntimeError("--max-new-tokens cannot be negative")
        ensure_model_files(
            args.cache_dir,
            no_download=args.no_download,
            timeout_seconds=args.timeout_seconds,
        )
        binary = build_lcqi_gpt2(args.build_dir, args.jobs)
        command = [
            str(binary),
            "--engine",
            args.engine,
            str(args.cache_dir),
```

```
            args.prompt,
            str(args.max_new_tokens),
        ]
        if args.benchmark:
            command.insert(1, "--benchmark")
        result = run_command(command, cwd=repo_dir(), timeout_seconds=args.timeout_seconds)
        write_report(
            args.report,
            cache_dir=args.cache_dir,
            binary=binary,
            prompt=args.prompt,
            max_new_tokens=args.max_new_tokens,
            engine=args.engine,
            result=result,
        )
        print(f"[lcqi-gpt2] report: {args.report}")
        print(tail(result.stdout, max_chars=2000))
        return 0 if result.returncode == 0 else result.returncode
    except Exception as error:
        print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Listing 16.7 tools/run_gpt2_benchmark_compare.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import os
import re
import shlex
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path

import run_gpt2_smoke


DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 4
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 300
DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smollm2-small-smoke"
SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"

```

```python

@dataclass(frozen=True)
class EngineRun:
    name: str
    command: list[str]
    returncode: int
    stdout: str
    stderr: str
    metrics: dict[str, str]


def volume_dir() -> Path:
    return Path(__file__).resolve().parents[1]


def repo_dir() -> Path:
    return Path(__file__).resolve().parents[3]


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def tail(text: str, max_chars: int = 4000) -> str:
    return text if len(text) <= max_chars else text[-max_chars:]


def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, str]:
    try:
        completed = subprocess.run(
            args,
            cwd=repo_dir(),
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return 124, stdout, stderr
    return completed.returncode, completed.stdout, completed.stderr

```

```python

def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    for line in stdout.splitlines():
        if not line:
            continue
        key, _, value = line.partition(" ")
        if key in {
            "engine",
            "generated_text",
            "benchmark_load_ms",
            "benchmark_tokenize_ms",
            "benchmark_prefill_ms",
            "benchmark_decode_ms",
            "benchmark_generate_ms",
            "benchmark_total_ms",
            "benchmark_prompt_tokens",
            "benchmark_generated_tokens",
            "benchmark_prefill_steps",
            "benchmark_decode_steps",
            "benchmark_generate_tokens_per_second",
            "benchmark_decode_tokens_per_second",
            "benchmark_kv_cache_bytes",
        }:
            metrics[key] = value
    return metrics


def run_lcqi(
    binary: Path,
    model_dir: Path,
    *,
    engine: str,
    prompt: str,
    max_new_tokens: int,
    timeout_seconds: int,
) -> EngineRun:
    command = [
        str(binary),
        "--benchmark",
        "--engine",
        engine,
        str(model_dir),
        prompt,
        str(max_new_tokens),
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=↩
    ↪ timeout_seconds)
    return EngineRun(
        name=f"lcqi_{engine}",
        command=command,
        returncode=returncode,
```

```
        stdout=stdout,
        stderr=stderr,
        metrics=parse_lcqi_metrics(stdout),
    )


def llama_bench_binary(cache_dir: Path) -> Path:
    return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"


def smollm2_model_path(cache_dir: Path) -> Path:
    return cache_dir / "models" / SMOLLM2_GGUF


def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    lines = [line.strip() for line in stdout.splitlines() if line.strip()]
    for line in lines:
        if not line.startswith("|"):
            continue
        columns = [column.strip() for column in line.strip("|").split("|")]
        if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(
    "-"):
            continue
        metrics["llama_model"] = columns[0]
        metrics["llama_size"] = columns[1]
        metrics["llama_params"] = columns[2]
        metrics["llama_backend"] = columns[3]
        metrics["llama_threads"] = columns[4]
        test_name = columns[5]
        tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
        if test_name.startswith("pp"):
            metrics["llama_prefill_test"] = test_name
            metrics["llama_prefill_tps"] = tokens_per_second
        elif test_name.startswith("tg"):
            metrics["llama_decode_test"] = test_name
            metrics["llama_decode_tps"] = tokens_per_second
    return metrics


def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun |
    None:
    binary = llama_bench_binary(cache_dir)
    model = smollm2_model_path(cache_dir)
    if not binary.exists() or not os.access(binary, os.X_OK) or not model.
    exists():
        return None
    command = [
        str(binary),
        "-m",
        str(model),
        "-ngl",
```

```
        "0",
        "-p",
        "5",
        "-n",
        "4",
        "-r",
        "1",
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=timeout_seconds)
    return EngineRun(
        name="llama_cpp_smollm2_q4_k_m",
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_llama_bench_metrics(stdout),
    )


def write_report(path: Path, runs: list[EngineRun], *, prompt: str, max_new_tokens: int) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Benchmark Compare",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"repo_commit={current_commit()}",
        f"working_tree_dirty={working_tree_dirty()}",
        f"python={sys.version.split()[0]}",
        f"prompt={prompt}",
        f"max_new_tokens={max_new_tokens}",
        "",
        "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 safetensors model. "
        "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature-engine reference, "
        "so its numbers are engineering context, not same-model quality ranking.",
        "",
    ]
    for run in runs:
        lines.extend(
            [
                f"[{run.name}]",
                f"returncode={run.returncode}",
                f"command={command_line(run.command)}",
            ]
        )
        for key in sorted(run.metrics):
            lines.append(f"{key}={run.metrics[key]}")
        lines.extend(
```

```
            [
                "stdout_tail:",
                tail(run.stdout.strip(), 2000),
                "stderr_tail:",
                tail(run.stderr.strip(), 2000),
                "",
            ]
        )
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def current_commit() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
        text=True,
        capture_output=True,
        check=False,
    )
    return result.stdout.strip() if result.returncode == 0 else "unknown"


def working_tree_dirty() -> str:
    result = subprocess.run(
        ["git", "-C", str(repo_dir()), "status", "--porcelain"],
        text=True,
        capture_output=True,
        check=False,
    )
    if result.returncode != 0:
        return "unknown"
    return "true" if result.stdout.strip() else "false"


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with optional llama.cpp context."
    )
    parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-llama", action="store_true")
```

```
    parser.add_argument("--smollm2-cache-dir", type=Path, default=DEFAULT_SMOLLM2_CACHE_DIR)
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.max_new_tokens < 0:
        print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.stderr)
        return 1
    try:
        run_gpt2_smoke.ensure_model_files(
            args.cache_dir,
            no_download=args.no_download,
            timeout_seconds=args.timeout_seconds,
        )
        binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
        runs = [
            run_lcqi(
                binary,
                args.cache_dir,
                engine="full",
                prompt=args.prompt,
                max_new_tokens=args.max_new_tokens,
                timeout_seconds=args.timeout_seconds,
            ),
            run_lcqi(
                binary,
                args.cache_dir,
                engine="cached",
                prompt=args.prompt,
                max_new_tokens=args.max_new_tokens,
                timeout_seconds=args.timeout_seconds,
            ),
        ]
        if not args.skip_llama:
            llama_run = run_llama_bench(
                args.smollm2_cache_dir.expanduser().resolve(),
                timeout_seconds=args.timeout_seconds,
            )
            if llama_run is not None:
                runs.append(llama_run)
        write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args.max_new_tokens)
    except Exception as error:
        print(f"[lcqi-compare] error: {error}", file=sys.stderr)
        return 1

    for run in runs:
        print(f"{run.name}: returncode={run.returncode}")
        for key in sorted(run.metrics):
```

```
            print(f"  {key}={run.metrics[key]}")
    print(f"report={args.report}")
    return 0 if all(run.returncode == 0 for run in runs) else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

二、公共合同头文件

Listing 16.8 include/lcqi/inference.hpp

```
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct InferenceResult {
    std::vector<float> logits;
    std::int32_t predicted_class = 0;
};

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output);

void relu_inplace(std::span<float> values);

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input);

}  // namespace lcqi
```

Listing 16.9 include/lcqi/int8_kernels.hpp

```
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct PackedLinearI8 {
```

```
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> packed_weights;
    std::vector<float> bias;
};

struct KernelBenchmarkCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    std::int32_t repeat = 0;
};

struct KernelBackendBenchmark {
    const char* name = "";
    bool available = false;
    double average_us = 0.0;
    float max_abs_diff = 0.0F;
    float checksum = 0.0F;
};

struct KernelBenchmarkResult {
    KernelBenchmarkCase benchmark_case;
    std::vector<KernelBackendBenchmark> backends;
};

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size);

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output);

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace);

bool linear_i8_packed_avx2_available() noexcept;

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output);

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale);

std::vector<float> make_deterministic_input(std::int32_t input_size);

```

```
float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& benchmark_case);

}  // namespace lcqi
```

Listing 16.10 include/lcqi/model.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <vector>

namespace lcqi {

struct QuantizedLinearLayer {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> weights;
    std::vector<float> bias;
};

struct TinyMlpModel {
    std::int32_t input_size = 0;
    std::int32_t hidden_size = 0;
    std::int32_t output_size = 0;
    QuantizedLinearLayer hidden;
    QuantizedLinearLayer output;
};

TinyMlpModel load_model(const std::filesystem::path& path);

}  // namespace lcqi
```

Listing 16.11 include/lcqi/reference_decoder.hpp

```
#pragma once

#include <cstdint>
#include <span>
#include <string>
#include <vector>

namespace lcqi {

struct LinearWeightsF32 {
    std::int32_t input_size = 0;
```

```
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct DecoderConfig {
    std::int32_t hidden_size = 0;
    std::int32_t query_heads = 0;
    std::int32_t kv_heads = 0;
    std::int32_t head_dim = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t vocab_size = 0;
    std::int32_t max_context = 0;
    float rms_epsilon = 1.0e-5F;
    float rope_base = 10000.0F;
};

struct DecoderLayerWeightsF32 {
    std::vector<float> rms_attention_weight;
    LinearWeightsF32 wq;
    LinearWeightsF32 wk;
    LinearWeightsF32 wv;
    LinearWeightsF32 wo;
    std::vector<float> rms_mlp_weight;
    LinearWeightsF32 w_gate;
    LinearWeightsF32 w_up;
    LinearWeightsF32 w_down;
};

struct ReferenceDecoderModel {
    DecoderConfig config;
    std::vector<float> token_embedding;
    std::vector<DecoderLayerWeightsF32> layers;
    std::vector<float> final_norm_weight;
    LinearWeightsF32 lm_head;
};

struct ReferenceDecodeResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct ReferenceTensorCheckpoint {
    std::string name;
    std::int32_t token_position = 0;
    std::int32_t layer_id = 0;
    std::vector<std::int32_t> shape;
    std::vector<std::int32_t> stride;
    std::string dtype;
    std::string layout;
    float checksum = 0.0F;
    float max_abs = 0.0F;
```

```
    std::vector<float> values;
};

struct ReferenceDecodeTraceResult {
    ReferenceDecodeResult result;
    std::vector<ReferenceTensorCheckpoint> checkpoints;
};

class ReferenceKVCache {
public:
    ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const;

    [[nodiscard]] std::int32_t layer_count() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;

private:
    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t kv_head) const;

    DecoderConfig config_;
    std::int32_t layer_count_ = 0;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output);

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output);
```

```
116
117 void apply_rope(std::span<float> heads,
118                  std::int32_t head_count,
119                  std::int32_t head_dim,
120                  std::int32_t model_position,
121                  float rope_base);
122
123 void attention_decode(const DecoderConfig& config,
124                       const ReferenceKVCache& cache,
125                       std::int32_t layer_id,
126                       std::int32_t model_position,
127                       std::span<const float> query,
128                       std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131                                            std::span<const std::int32_t> ↩
    ↪ token_ids);
132
133 ReferenceDecodeTraceResult run_reference_decode_with_trace(
134     const ReferenceDecoderModel& model,
135     std::span<const std::int32_t> token_ids);
136
137 ReferenceDecoderModel make_tiny_reference_decoder_model();
138
139 }  // namespace lcqi
```

Listing 16.12 include/lcqi/gpt2_reference.hpp

```
1  #pragma once
2
3  #include <cstdint>
4  #include <filesystem>
5  #include <span>
6  #include <string>
7  #include <string_view>
8  #include <unordered_map>
9  #include <vector>
10
11 namespace lcqi {
12
13 struct Gpt2Config {
14     std::int32_t hidden_size = 0;
15     std::int32_t head_count = 0;
16     std::int32_t layer_count = 0;
17     std::int32_t max_positions = 0;
18     std::int32_t vocab_size = 0;
19     std::int32_t intermediate_size = 0;
20     std::int32_t bos_token_id = -1;
21     std::int32_t eos_token_id = -1;
22     float layer_norm_epsilon = 1.0e-5F;
23     std::string activation_function = "gelu_new";
24 };
```

```
25
26 struct Gpt2LinearF32 {
27     std::int32_t input_size = 0;
28     std::int32_t output_size = 0;
29     std::vector<float> weights;
30     std::vector<float> bias;
31 };
32
33 struct Gpt2LayerWeightsF32 {
34     std::vector<float> ln_1_weight;
35     std::vector<float> ln_1_bias;
36     Gpt2LinearF32 c_attn;
37     Gpt2LinearF32 c_proj;
38     std::vector<float> ln_2_weight;
39     std::vector<float> ln_2_bias;
40     Gpt2LinearF32 c_fc;
41     Gpt2LinearF32 mlp_c_proj;
42 };
43
44 struct Gpt2ReferenceModel {
45     Gpt2Config config;
46     std::vector<float> token_embedding;
47     std::vector<float> position_embedding;
48     std::vector<Gpt2LayerWeightsF32> layers;
49     std::vector<float> final_ln_weight;
50     std::vector<float> final_ln_bias;
51     std::vector<float> lm_head_weight;
52     bool tie_lm_head_to_embedding = true;
53 };
54
55 struct Gpt2ForwardResult {
56     std::vector<float> logits;
57     std::int32_t predicted_token = 0;
58 };
59
60 class Gpt2KvCache {
61 public:
62     explicit Gpt2KvCache(const Gpt2Config& config);
63
64     void append(std::int32_t layer_id,
65                 std::int32_t model_position,
66                 std::span<const float> key,
67                 std::span<const float> value);
68
69     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
70                                              std::int32_t model_position,
71                                              std::int32_t head) const;
72
73     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
74                                                std::int32_t model_position,
75                                                std::int32_t head) const;
76
```

```
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
    [[nodiscard]] std::size_t byte_size() const noexcept;

private:
    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t head) const;
    void validate_written(std::int32_t layer_id,
                          std::int32_t model_position) const;

    Gpt2Config config_;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

struct Gpt2Tokenizer {
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    std::unordered_map<std::string, std::int32_t> vocab;
    std::vector<std::string> id_to_token;
    std::unordered_map<std::string, std::int32_t> bpe_ranks;
};

[[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
    const std::filesystem::path& vocab_json,
    const std::filesystem::path& merges_txt,
    std::int32_t bos_token_id,
    std::int32_t eos_token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_encode(
    const Gpt2Tokenizer& tokenizer,
    std::string_view text);

[[nodiscard]] std::string gpt2_decode(
    const Gpt2Tokenizer& tokenizer,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& config_json);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
    const std::filesystem::path& config_json,
    const std::filesystem::path& safetensors_path);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
    const std::filesystem::path& model_directory);

```

```
[[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
    const Gpt2ReferenceModel& model,
    Gpt2KvCache& cache,
    std::int32_t token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens);

[[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens);

[[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();

}  // namespace lcqi
```

Listing 16.13 include/lcqi/safetensors.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <span>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

namespace lcqi {

struct SafeTensorEntry {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::uint64_t data_begin = 0;
    std::uint64_t data_end = 0;

    [[nodiscard]] std::uint64_t byte_size() const;
};

struct SafeTensorManifest {
    std::uint64_t header_size = 0;
    std::uint64_t data_start_offset = 0;
    std::vector<std::pair<std::string, std::string>> metadata;
    std::vector<SafeTensorEntry> tensors;
```

```
28
29      [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↩
        ↪ const;
30  };
31
32  struct SafeTensorFile {
33      std::filesystem::path path;
34      SafeTensorManifest manifest;
35      std::vector<std::uint8_t> bytes;
36  };
37
38  [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
39      const std::filesystem::path& path);
40
41  [[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path&↩
        ↪  path);
42
43  [[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
44      const SafeTensorFile& file,
45      std::string_view name);
46
47  [[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
48      const SafeTensorFile& file,
49      std::string_view name);
50
51  }  // namespace lcqi
```

Listing 16.14 include/lcqi/tokenizer.hpp

```
1  #pragma once
2
3  #include <cstdint>
4  #include <filesystem>
5  #include <string>
6  #include <string_view>
7  #include <vector>
8
9  namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
```

```
[[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);

[[nodiscard]] std::vector<std::int32_t> encode_prompt(
    const TokenizerModel& tokenizer,
    std::string_view prompt);

[[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer);

}  // namespace lcqi
```

三、核心实现

Listing 16.15 src/inference.cpp

```
#include <lcqi/inference.hpp>

#include <algorithm>
#include <cmath>
#include <stdexcept>

namespace lcqi {
namespace {

void validate_input_size(std::span<const float> input, std::int32_t expected) {
    if (input.size() != static_cast<std::size_t>(expected)) {
        throw std::runtime_error("input size does not match model shape");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i) {
        const float candidate = values[static_cast<std::size_t>(i)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = i;
        }
    }
    return best_index;
}

}  // namespace

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
```

```cpp
                std::span<float> output) {
    if (input.size() != static_cast<std::size_t>(layer.input_size)) {
        throw std::runtime_error("linear_i8 input size mismatch");
    }
    if (output.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error("linear_i8 output size mismatch");
    }

    for (std::int32_t out = 0; out < layer.output_size; ++out) {
        const std::size_t row_base =
            static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.input_size);
        float acc = layer.bias[static_cast<std::size_t>(out)];
        for (std::int32_t in = 0; in < layer.input_size; ++in) {
            const std::int8_t weight =
                layer.weights[row_base + static_cast<std::size_t>(in)];
            acc += static_cast<float>(weight) * layer.scale *
                   input[static_cast<std::size_t>(in)];
        }
        output[static_cast<std::size_t>(out)] = acc;
    }
}

void relu_inplace(std::span<float> values) {
    for (float& value : values) {
        value = std::max(0.0F, value);
    }
}

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input) {
    validate_input_size(input, model.input_size);

    std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F);
    std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F);

    linear_i8(model.hidden, input, hidden);
    relu_inplace(hidden);
    linear_i8(model.output, hidden, logits);

    InferenceResult result;
    result.predicted_class = argmax(logits);
    result.logits = std::move(logits);
    return result;
}

}  // namespace lcqi
```

Listing 16.16 src/int8_kernels.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#include <lcqi/inference.hpp>

#include <algorithm>
#include <chrono>
#include <cmath>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
constexpr const char* LCQI_BACKEND_AVX2 = "avx2";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

void validate_quantized_layer(const QuantizedLinearLayer& layer) {
    require_positive(layer.input_size, "input_size");
    require_positive(layer.output_size, "output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, "input_size") *
        checked_size(layer.output_size, "output_size");
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error("quantized layer weight size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("quantized layer bias size mismatch");
    }
}

```

```cpp
std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
    result.backends.push_back(KernelBackendBenchmark{
        .name = name,
        .available = false,
        .average_us = 0.0,
        .max_abs_diff = 0.0F,
        .checksum = 0.0F,
    });
}

template <typename Callable>
double measure_average_us(std::int32_t repeat, Callable&& callable) {
    require_positive(repeat, "repeat");
    const auto begin = std::chrono::steady_clock::now();
    for (std::int32_t index = 0; index < repeat; ++index) {
        callable();
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed = end - begin;
    return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
}

}  // namespace

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size) {
    validate_quantized_layer(layer);
    require_positive(output_block_size, "output_block_size");

    PackedLinearI8 packed;
    packed.input_size = layer.input_size;
    packed.output_size = layer.output_size;
    packed.output_block_size = output_block_size;
    packed.scale = layer.scale;
    packed.bias = layer.bias;

    const std::int32_t padded_outputs =
```

```
        round_up_to_block(layer.output_size, output_block_size);
    packed.packed_weights.assign(
        checked_size(padded_outputs, "padded_outputs") *
            checked_size(layer.input_size, "input_size"),
        0);

    std::size_t packed_index = 0;
    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += output_block_size) {
        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            for (std::int32_t offset = 0; offset < output_block_size; ++offset) {
                const std::int32_t output_index = output_block + offset;
                if (output_index < layer.output_size) {
                    const std::size_t source =
                        checked_size(output_index, "output_index") *
                            checked_size(layer.input_size, "input_size") +
                        checked_size(input_index, "input_index");
                    packed.packed_weights[packed_index] = layer.weights[source];
                }
                ++packed_index;
            }
        }
    }
    return packed;
}

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output) {
    std::vector<float> accumulators(
        checked_size(layer.output_block_size, "output_block_size"),
        0.0F);
    linear_i8_packed_with_workspace(layer, input, output, accumulators);
}

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    require_positive(layer.output_block_size, "packed output_block_size");
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("linear_i8_packed input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
```

```
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");
161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch"↩
    ↪ );
163     }
164     if (accumulator_workspace.size() != checked_size(layer.output_block_size, "↩
    ↪ output_block_size")) {
165         throw std::runtime_error("linear_i8_packed accumulator workspace size ↩
    ↪ mismatch");
166     }
167
168     std::size_t packed_index = 0;
169     for (std::int32_t output_block = 0; output_block < padded_outputs;
170          output_block += layer.output_block_size) {
171         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), ↩
    ↪ 0.0F);
172         for (std::int32_t offset = 0; offset < layer.output_block_size; ++↩
    ↪ offset) {
173             const std::int32_t output_index = output_block + offset;
174             if (output_index < layer.output_size) {
175                 accumulator_workspace[checked_size(offset, "offset")] =
176                     layer.bias[checked_size(output_index, "output_index")];
177             }
178         }
179
180         for (std::int32_t input_index = 0; input_index < layer.input_size; ++↩
    ↪ input_index) {
181             const float input_value = input[checked_size(input_index, "↩
    ↪ input_index")];
182             for (std::int32_t offset = 0; offset < layer.output_block_size; ++↩
    ↪ offset) {
183                 const std::int32_t output_index = output_block + offset;
184                 const std::int8_t weight = layer.packed_weights[packed_index↩
    ↪ ++];
185                 if (output_index < layer.output_size) {
186                     accumulator_workspace[checked_size(offset, "offset")] +=
187                         static_cast<float>(weight) * layer.scale * input_value;
188                 }
189             }
190         }
191
192         for (std::int32_t offset = 0; offset < layer.output_block_size; ++↩
    ↪ offset) {
193             const std::int32_t output_index = output_block + offset;
194             if (output_index < layer.output_size) {
```

```
                output[checked_size(output_index, "output_index")] =
                    accumulator_workspace[checked_size(offset, "offset")];
            }
        }
    }
}

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale) {
    require_positive(input_size, "input_size");
    require_positive(output_size, "output_size");
    QuantizedLinearLayer layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.scale = scale;
    const std::size_t weight_count =
        checked_size(input_size, "input_size") * checked_size(output_size, "output_size");
    layer.weights.resize(weight_count);
    layer.bias.resize(checked_size(output_size, "output_size"));
    for (std::int32_t output_index = 0; output_index < output_size; ++output_index) {
        layer.bias[checked_size(output_index, "output_index")] =
            static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
        for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
            const std::int32_t raw =
                ((output_index * input_size + input_index) % LCQI_I8_PATTERN_MODULUS) -
                LCQI_I8_PATTERN_CENTER;
            layer.weights[checked_size(output_index, "output_index") *
                              checked_size(input_size, "input_size") +
                          checked_size(input_index, "input_index")] =
                static_cast<std::int8_t>(raw);
        }
    }
    return layer;
}

std::vector<float> make_deterministic_input(std::int32_t input_size) {
    require_positive(input_size, "input_size");
    std::vector<float> input(checked_size(input_size, "input_size"));
    for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
        input[checked_size(input_index, "input_index")] =
            static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
    }
```

```
    return input;
}

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
    if (lhs.size() != rhs.size()) {
        throw std::runtime_error("max_abs_diff size mismatch");
    }
    float diff = 0.0F;
    for (std::size_t index = 0; index < lhs.size(); ++index) {
        diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
    }
    return diff;
}

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& benchmark_case) {
    require_positive(benchmark_case.input_size, "benchmark input_size");
    require_positive(benchmark_case.output_size, "benchmark output_size");
    require_positive(benchmark_case.output_block_size, "benchmark output_block_size");
    require_positive(benchmark_case.repeat, "benchmark repeat");

    const QuantizedLinearLayer layer =
        make_deterministic_i8_layer(benchmark_case.input_size,
                                    benchmark_case.output_size,
                                    0.015625F);
    const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.output_block_size);
    const PackedLinearI8 packed_simd = pack_linear_i8(layer, LCQI_SIMD_OUTPUT_BLOCK);
    const std::vector<float> input = make_deterministic_input(benchmark_case.input_size);
    std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "output_size"),
                                     0.0F);
    std::vector<float> packed_output(scalar_output.size(), 0.0F);
    std::vector<float> simd_output(scalar_output.size(), 0.0F);
    std::vector<float> packed_workspace(
        checked_size(benchmark_case.output_block_size, "output_block_size"),
        0.0F);

    linear_i8(layer, input, scalar_output);
    linear_i8_packed_with_workspace(packed, input, packed_output, packed_workspace);

    KernelBenchmarkResult result;
    result.benchmark_case = benchmark_case;
    result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
    result.backends.push_back(KernelBackendBenchmark{
        .name = LCQI_BACKEND_SCALAR,
        .available = true,
        .average_us = measure_average_us(benchmark_case.repeat,
```

```
                                            [&]() { linear_i8(layer, input, scalar_output); }),
        .max_abs_diff = 0.0F,
        .checksum = checksum(scalar_output),
    });
    result.backends.push_back(KernelBackendBenchmark{
        .name = LCQI_BACKEND_PACKED_SCALAR,
        .available = true,
        .average_us = measure_average_us(
            benchmark_case.repeat,
            [&]() { linear_i8_packed_with_workspace(packed,
                                                    input,
                                                    packed_output,
                                                    packed_workspace); }),
        .max_abs_diff = max_abs_diff(scalar_output, packed_output),
        .checksum = checksum(packed_output),
    });
    if (linear_i8_packed_avx2_available()) {
        linear_i8_packed_avx2(packed_simd, input, simd_output);
        result.backends.push_back(KernelBackendBenchmark{
            .name = LCQI_BACKEND_AVX2,
            .available = true,
            .average_us = measure_average_us(
                benchmark_case.repeat,
                [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); }),
            .max_abs_diff = max_abs_diff(scalar_output, simd_output),
            .checksum = checksum(simd_output),
        });
    } else {
        add_unavailable_backend(result, LCQI_BACKEND_AVX2);
    }
    return result;
}

}  // namespace lcqi

#if !defined(LCQI_ENABLE_AVX2)
namespace lcqi {

bool linear_i8_packed_avx2_available() noexcept {
    return false;
}

void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
    throw std::runtime_error("AVX2 packed kernel is not enabled in this build");
}

}  // namespace lcqi
#endif
```

Listing 16.17 src/int8_kernels_avx2.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#if defined(LCQI_ENABLE_AVX2)

#include <immintrin.h>

#include <algorithm>
#include <cstdint>
#include <cstring>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

void validate_avx2_contract(const PackedLinearI8& layer,
                            std::span<const float> input,
                            std::span<float> output) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
        throw std::runtime_error("AVX2 packed kernel requires output_block_size == 8");
    }
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("AVX2 packed kernel input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel output size mismatch");
    }
```

```
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("AVX2 packed kernel bias size mismatch");
    }
    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("AVX2 packed kernel packed weight size mismatch");
    }
}

}  // namespace

bool linear_i8_packed_avx2_available() noexcept {
#if defined(__GNUC__) || defined(__clang__)
    __builtin_cpu_init();
    return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
#else
    return true;
#endif
}

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output) {
    if (!linear_i8_packed_avx2_available()) {
        throw std::runtime_error("AVX2/FMA is not available on this CPU");
    }
    validate_avx2_contract(layer, input, output);

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    std::size_t packed_index = 0;
    alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};

    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
        __m256 accumulator = _mm256_setzero_ps();

        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
            const std::int32_t output_index = output_block + offset;
            block_output[checked_size(offset, "offset")] =
                output_index < layer.output_size
                    ? layer.bias[checked_size(output_index, "output_index")]
                    : 0.0F;
        }
        accumulator = _mm256_load_ps(block_output);

```

```
        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            std::uint64_t packed_bytes = 0;
            std::memcpy(&packed_bytes,
                        layer.packed_weights.data() + packed_index,
                        sizeof(packed_bytes));
            const __m128i packed_i8 =
                _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
            packed_index += LCQI_AVX2_OUTPUT_BLOCK;
            const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
            const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
            const __m256 input_scaled =
                _mm256_set1_ps(input[checked_size(input_index, "input_index")] * layer.scale);
            accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, accumulator);
        }

        _mm256_store_ps(block_output, accumulator);
        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset ) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    block_output[checked_size(offset, "offset")];
            }
        }
    }
}

}  // namespace lcqi

#endif
```

Listing 16.18 src/model.cpp

```
#include <lcqi/model.hpp>

#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
```

```
        throw std::runtime_error("unexpected end of model file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid int32 value for key '" + key + "'");
    }
    return value;
}

float read_float(std::istream& input, const std::string& key) {
    expect_key(input, key);
    float value = 0.0F;
    if (!(input >> value)) {
        throw std::runtime_error("invalid float value for key '" + key + "'");
    }
    return value;
}

std::vector<std::int8_t> read_i8_vector(std::istream& input,
                                        const std::string& key,
                                        std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<std::int8_t> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        std::int32_t raw = 0;
        if (!(input >> raw)) {
            throw std::runtime_error("invalid int8 payload for key '" + key + "'");
        }
        if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
            throw std::runtime_error("int8 payload out of range for key '" + key + "'");
        }
        values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
    }
    return values;
```

```
}

std::vector<float> read_float_vector(std::istream& input,
                                     const std::string& key,
                                     std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<float> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        if (!(input >> values[static_cast<std::size_t>(i)])) {
            throw std::runtime_error("invalid float payload for key '" + key + "'");
        }
    }
    return values;
}

void validate_layer(const QuantizedLinearLayer& layer, const std::string& name) {
    if (layer.input_size <= 0 || layer.output_size <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::size_t expected_weights =
        static_cast<std::size_t>(layer.input_size) *
        static_cast<std::size_t>(layer.output_size);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(name + " layer weight size mismatch");
    }
    if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error(name + " layer bias size mismatch");
    }
}

std::int32_t checked_element_count(std::int32_t rows,
                                   std::int32_t columns,
                                   const std::string& name) {
    if (rows <= 0 || columns <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::int64_t count =
        static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
    if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::max())) {
        throw std::runtime_error(name + " layer element count is too large");
    }
    return static_cast<std::int32_t>(count);
}

}  // namespace

```

```
TinyMlpModel load_model(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open model file: " + path.string());
    }

    expect_key(input, "LCQI_MODEL_V1");

    TinyMlpModel model;
    model.input_size = read_i32(input, "input_size");
    model.hidden_size = read_i32(input, "hidden_size");
    model.output_size = read_i32(input, "output_size");

    const std::int32_t hidden_weight_count =
        checked_element_count(model.input_size, model.hidden_size, "hidden");
    const std::int32_t output_weight_count =
        checked_element_count(model.hidden_size, model.output_size, "output");

    model.hidden.input_size = model.input_size;
    model.hidden.output_size = model.hidden_size;
    model.hidden.scale = read_float(input, "hidden_scale");
    model.hidden.weights = read_i8_vector(
        input,
        "hidden_weights",
        hidden_weight_count);
    model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);

    model.output.input_size = model.hidden_size;
    model.output.output_size = model.output_size;
    model.output.scale = read_float(input, "output_scale");
    model.output.weights = read_i8_vector(
        input,
        "output_weights",
        output_weight_count);
    model.output.bias = read_float_vector(input, "output_bias", model.output_size);

    validate_layer(model.hidden, "hidden");
    validate_layer(model.output, "output");
    return model;
}

}  // namespace lcqi
```

Listing 16.19 src/reference_decoder.cpp

```
#include <lcqi/reference_decoder.hpp>

#include <algorithm>
#include <array>
#include <cmath>
```

```cpp
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const DecoderConfig& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.query_heads, "query_heads");
    require_positive(config.kv_heads, "kv_heads");
    require_positive(config.head_dim, "head_dim");
    require_positive(config.intermediate_size, "intermediate_size");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.max_context, "max_context");
    if (config.query_heads % config.kv_heads != 0) {
        throw std::runtime_error("query_heads must be divisible by kv_heads");
    }
    if (config.hidden_size != config.query_heads * config.head_dim) {
        throw std::runtime_error("hidden_size must equal query_heads * head_dim");
    }
    if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("head_dim must be even for RoPE");
    }
    if (config.rope_base <= 1.0F) {
        throw std::runtime_error("rope_base must be greater than 1");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::int32_t kv_width(const DecoderConfig& config) {
```

```cpp
    return config.kv_heads * config.head_dim;
}

void validate_linear(const LinearWeightsF32& layer, const char* name) {
    require_positive(layer.input_size, "linear input_size");
    require_positive(layer.output_size, "linear output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, name) * checked_size(layer.output_size, name);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(std::string(name) + " weight size mismatch");
    }
    if (!layer.bias.empty() &&
        layer.bias.size() != checked_size(layer.output_size, name)) {
        throw std::runtime_error(std::string(name) + " bias size mismatch");
    }
}

void validate_span_size(std::size_t actual, std::int32_t expected, const char* name) {
    if (actual != checked_size(expected, name)) {
        throw std::runtime_error(std::string(name) + " size mismatch");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty values");
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()); ++index) {
        const float candidate = values[static_cast<std::size_t>(index)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = index;
        }
    }
    return best_index;
}

float silu(float value) {
    return value / (1.0F + std::exp(-value));
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    const std::size_t begin = checked_size(row, "row") * checked_size(row_width, "row_width");
    return values.subspan(begin, checked_size(row_width, "row_width"));
```

```
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void swiglu_mlp(const DecoderLayerWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    linear_f32(layer.w_down, hidden, output);
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

float max_abs(std::span<const float> values) {
    float result = 0.0F;
    for (const float value : values) {
        result = std::max(result, std::fabs(value));
    }
    return result;
}

std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape) {
    std::vector<std::int32_t> stride(shape.size(), 1);
    std::int32_t running = 1;
    for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; index >= 0; --index) {
        stride[checked_size(index, "stride index")] = running;
```

```
153         running *= shape[checked_size(index, "shape index")];
154     }
155     return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159                     const char* name,
160                     std::int32_t token_position,
161                     std::int32_t layer_id,
162                     std::span<const std::int32_t> shape,
163                     const char* layout,
164                     std::span<const float> values) {
165     if (checkpoints == nullptr) {
166         return;
167     }
168     ReferenceTensorCheckpoint checkpoint;
169     checkpoint.name = name;
170     checkpoint.token_position = token_position;
171     checkpoint.layer_id = layer_id;
172     checkpoint.shape.assign(shape.begin(), shape.end());
173     checkpoint.stride = contiguous_stride(shape);
174     checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175     checkpoint.layout = layout;
176     checkpoint.checksum = checksum(values);
177     checkpoint.max_abs = max_abs(values);
178     checkpoint.values.assign(values.begin(), values.end());
179     checkpoints->push_back(std::move(checkpoint));
180 }
181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                            std::span<const float> input,
184                            std::span<float> output,
185                            std::int32_t token_position,
186                            std::int32_t layer_id,
187                            std::vector<ReferenceTensorCheckpoint>* checkpoints)↩
    ↪  {
188     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0↩
    ↪ F);
189     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
190     std::vector<float> hidden(gate.size(), 0.0F);
191     linear_f32(layer.w_gate, input, gate);
192     linear_f32(layer.w_up, input, up);
193     if (gate.size() != up.size()) {
194         throw std::runtime_error("SwiGLU gate/up size mismatch");
195     }
196     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↩
    ↪ output_size};
197     add_checkpoint(checkpoints,
198                    "mlp_gate",
199                    token_position,
200                    layer_id,
201                    intermediate_shape,
```

```
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    gate);
    add_checkpoint(checkpoints,
                    "mlp_up",
                    token_position,
                    layer_id,
                    intermediate_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    up);
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    add_checkpoint(checkpoints,
                    "mlp_hidden",
                    token_position,
                    layer_id,
                    intermediate_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    hidden);
    linear_f32(layer.w_down, hidden, output);
}

void forward_layer(const DecoderConfig& config,
                    const DecoderLayerWeightsF32& layer,
                    std::int32_t layer_id,
                    std::int32_t model_position,
                    ReferenceKVCache& cache,
                    std::span<float> hidden,
                    std::vector<ReferenceTensorCheckpoint>* checkpoints = ↩
    ↪ nullptr) {
    validate_span_size(hidden.size(), config.hidden_size, "hidden");

    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), ↩
    ↪ 0.0F);
    std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), ↩
    ↪ 0.0F);
    std::vector<float> key(checked_size(kv_width(config), "kv_width"), 0.0F);
    std::vector<float> value(checked_size(kv_width(config), "kv_width"), 0.0F);
    std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"↩
    ↪ ), 0.0F);
    std::vector<float> attention_projected(checked_size(config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
    std::vector<float> mlp(checked_size(config.hidden_size, "hidden_size"), 0.0↩
    ↪ F);

    const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
    const std::array<std::int32_t, 2> query_shape{config.query_heads, config.↩
    ↪ head_dim};
    const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim↩
    ↪ };
    const std::array<std::int32_t, 4> kv_slot_shape{
        1, 1, config.kv_heads, config.head_dim};
```

```
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249                    "attn_norm",
250                    model_position,
251                    layer_id,
252                    hidden_shape,
253                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
254                    normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259                    "q_before_rope",
260                    model_position,
261                    layer_id,
262                    query_shape,
263                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
264                    query);
265     add_checkpoint(checkpoints,
266                    "k_before_rope",
267                    model_position,
268                    layer_id,
269                    kv_shape,
270                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
271                    key);
272     add_checkpoint(checkpoints,
273                    "v",
274                    model_position,
275                    layer_id,
276                    kv_shape,
277                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
278                    value);
279     apply_rope(query, config.query_heads, config.head_dim, model_position, ↩
    ↪ config.rope_base);
280     apply_rope(key, config.kv_heads, config.head_dim, model_position, config.↩
    ↪ rope_base);
281     add_checkpoint(checkpoints,
282                    "q_after_rope",
283                    model_position,
284                    layer_id,
285                    query_shape,
286                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
287                    query);
288     add_checkpoint(checkpoints,
289                    "k_after_rope",
290                    model_position,
291                    layer_id,
292                    kv_shape,
293                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
294                    key);
295
```

```
    cache.append(layer_id, model_position, key, value);
    add_checkpoint(checkpoints,
                   "kv_cache_key_slot",
                   model_position,
                   layer_id,
                   kv_slot_shape,
                   LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
                   key);
    attention_decode(config, cache, layer_id, model_position, query, attention);
    add_checkpoint(checkpoints,
                   "attention_out",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   attention);
    linear_f32(layer.wo, attention, attention_projected);
    add_inplace(hidden, attention_projected);
    add_checkpoint(checkpoints,
                   "post_attention_residual",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);

    rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
    add_checkpoint(checkpoints,
                   "mlp_norm",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    if (checkpoints == nullptr) {
        swiglu_mlp(layer, normed, mlp);
    } else {
        swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, checkpoints);
    }
    add_inplace(hidden, mlp);
    add_checkpoint(checkpoints,
                   "layer_out",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   hidden);
}

LinearWeightsF32 make_linear(std::int32_t input_size,
```

```
                              std::int32_t output_size,
                              std::vector<float> weights,
                              std::vector<float> bias = {}) {
    LinearWeightsF32 layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.weights = std::move(weights);
    layer.bias = std::move(bias);
    validate_linear(layer, "tiny linear");
    return layer;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

}  // namespace

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids) {
    validate_config(model.config);
    if (token_ids.empty()) {
        throw std::runtime_error("reference decode needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_context, "max_context"
    )) {
        throw std::runtime_error("token_ids exceed max_context");
    }
    if (model.token_embedding.size() !=
        checked_size(model.config.vocab_size, "vocab_size") *
            checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("token embedding size mismatch");
    }
    if (model.final_norm_weight.size() != checked_size(model.config.hidden_size
    , "hidden_size")) {
        throw std::runtime_error("final norm weight size mismatch");
    }
    validate_linear(model.lm_head, "lm_head");

    ReferenceDecodeTraceResult trace;
    ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers
    .size()));
    std::vector<float> hidden(checked_size(model.config.hidden_size, "
    hidden_size"), 0.0F);
    const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
    for (std::int32_t position = 0; position < static_cast<std::int32_t>(
    token_ids.size());
         ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "
    position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
```

```
            throw std::runtime_error("token id out of vocabulary");
        }
        const std::span<const float> embedding =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        std::copy(embedding.begin(), embedding.end(), hidden.begin());
        add_checkpoint(&trace.checkpoints,
                       "embedding",
                       position,
                       -1,
                       hidden_shape,
                       LCQI_TRACE_LAYOUT_ROW_MAJOR,
                       hidden);
        for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(model.layers.size());
             ++layer_id) {
            forward_layer(model.config,
                          model.layers[checked_size(layer_id, "layer_id")],
                          layer_id,
                          position,
                          cache,
                          hidden,
                          &trace.checkpoints);
        }
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
    const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size()) - 1;
    rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed);
    add_checkpoint(&trace.checkpoints,
                   "final_norm",
                   last_position,
                   -1,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    linear_f32(model.lm_head, normed, logits);
    add_checkpoint(&trace.checkpoints,
                   "logits",
                   last_position,
                   -1,
                   logits_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   logits);

    trace.result.predicted_token = argmax(logits);
```

```
    trace.result.logits = std::move(logits);
    return trace;
}

ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count)
    : config_(config),
      layer_count_(layer_count) {
    validate_config(this->config_);
    require_positive(this->layer_count_, "layer_count");
    const std::size_t element_count =
        checked_size(this->layer_count_, "layer_count") *
        checked_size(this->config_.max_context, "max_context") *
        checked_size(this->config_.kv_heads, "kv_heads") *
        checked_size(this->config_.head_dim, "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->layer_count_, "layer_count") *
            checked_size(this->config_.max_context, "max_context"),
        0);
}

void ReferenceKVCache::append(std::int32_t layer_id,
                              std::int32_t model_position,
                              std::span<const float> key,
                              std::span<const float> value) {
    validate_span_size(key.size(), kv_width(this->config_), "KV key");
    validate_span_size(value.size(), kv_width(this->config_), "KV value");
    this->validate_address(layer_id, model_position, 0);
    for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
        const std::size_t target = this->base_offset(layer_id, model_position, kv_head);
        const std::size_t source =
            checked_size(kv_head, "kv_head") * checked_size(this->config_.head_dim, "head_dim");
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") * checked_size(this->config_.max_context, "max_context") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}
```

```
484
485 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
486                                               std::int32_t model_position,
487                                               std::int32_t kv_head) const {
488     this->validate_address(layer_id, model_position, kv_head);
489     const std::size_t offset = this->base_offset(layer_id, model_position, ↩
        ↪ kv_head);
490     return std::span<const float>(this->keys_.data() + offset,
491                                   checked_size(this->config_.head_dim, "↩
        ↪ head_dim"));
492 }
493
494 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
495                                                 std::int32_t model_position,
496                                                 std::int32_t kv_head) const {
497     this->validate_address(layer_id, model_position, kv_head);
498     const std::size_t offset = this->base_offset(layer_id, model_position, ↩
        ↪ kv_head);
499     return std::span<const float>(this->values_.data() + offset,
500                                   checked_size(this->config_.head_dim, "↩
        ↪ head_dim"));
501 }
502
503 std::int32_t ReferenceKVCache::layer_count() const noexcept {
504     return this->layer_count_;
505 }
506
507 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
508     return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512                                           std::int32_t model_position,
513                                           std::int32_t kv_head) const {
514     return (((checked_size(layer_id, "layer_id") *
515               checked_size(this->config_.max_context, "max_context")) +
516              checked_size(model_position, "model_position")) *
517                 checked_size(this->config_.kv_heads, "kv_heads") +
518             checked_size(kv_head, "kv_head")) *
519            checked_size(this->config_.head_dim, "head_dim");
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523                                         std::int32_t model_position,
524                                         std::int32_t kv_head) const {
525     if (layer_id < 0 || layer_id >= this->layer_count_) {
526         throw std::runtime_error("KV layer_id out of range");
527     }
528     if (model_position < 0 || model_position >= this->config_.max_context) {
529         throw std::runtime_error("KV model_position out of range");
530     }
531     if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
```

```
532         throw std::runtime_error("KV head out of range");
533     }
534 }
535
536 void rms_norm(std::span<const float> input,
537               std::span<const float> weight,
538               float epsilon,
539               std::span<float> output) {
540     if (input.size() != weight.size() || input.size() != output.size()) {
541         throw std::runtime_error("RMSNorm size mismatch");
542     }
543     if (input.empty()) {
544         throw std::runtime_error("RMSNorm input is empty");
545     }
546     float sum_square = 0.0F;
547     for (const float value : input) {
548         sum_square += value * value;
549     }
550     const float mean_square = sum_square / static_cast<float>(input.size());
551     const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552     for (std::size_t index = 0; index < input.size(); ++index) {
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556
557 void linear_f32(const LinearWeightsF32& layer,
558                 std::span<const float> input,
559                 std::span<float> output) {
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↩
    ↪ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↩
    ↪ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                            input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                 std::int32_t head_count,
577                 std::int32_t head_dim,
578                 std::int32_t model_position,
579                 float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
```

```
    validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
    if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
        throw std::runtime_error("RoPE head_dim must be even");
    }
    if (model_position < 0) {
        throw std::runtime_error("RoPE model_position cannot be negative");
    }
    if (rope_base <= 1.0F) {
        throw std::runtime_error("RoPE base must be greater than 1");
    }
    for (std::int32_t head = 0; head < head_count; ++head) {
        for (std::int32_t offset = 0; offset < head_dim; offset += LCQI_ROPE_PAIR_WIDTH) {
            const float exponent = static_cast<float>(offset) / static_cast<float>(head_dim);
            const float theta = 1.0F / std::pow(rope_base, exponent);
            const float angle = static_cast<float>(model_position) * theta;
            const float cosine = std::cos(angle);
            const float sine = std::sin(angle);
            const std::size_t first =
                checked_size(head, "head") * checked_size(head_dim, "head_dim") +
                checked_size(offset, "offset");
            const float x0 = heads[first];
            const float x1 = heads[first + 1];
            heads[first] = x0 * cosine - x1 * sine;
            heads[first + 1] = x0 * sine + x1 * cosine;
        }
    }
}

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output) {
    validate_config(config);
    validate_span_size(query.size(), config.hidden_size, "attention query");
    validate_span_size(output.size(), config.hidden_size, "attention output");
    if (model_position < 0 || model_position >= cache.filled_tokens()) {
        throw std::runtime_error("attention model_position has not been written");
    }
    std::fill(output.begin(), output.end(), 0.0F);

    const std::int32_t group_size = config.query_heads / config.kv_heads;
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.head_dim));
    std::vector<float> scores(checked_size(model_position + 1, "score count"),
                              LCQI_SOFTMAX_NEGATIVE_INFINITY);
    std::vector<float> probabilities(scores.size(), 0.0F);
```

```
629
630     for (std::int32_t query_head = 0; query_head < config.query_heads; ++query_head) {
631         const std::int32_t kv_head = query_head / group_size;
632         const std::size_t query_base =
633             checked_size(query_head, "query_head") * checked_size(config.head_dim, "head_dim");
634         float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
635         for (std::int32_t position = 0; position <= model_position; ++position) {
636             const std::span<const float> key = cache.key(layer_id, position, kv_head);
637             float dot = 0.0F;
638             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
639                 dot += query[query_base + checked_size(dim, "dim")] *
640                        key[checked_size(dim, "dim")];
641             }
642             const float score = dot * score_scale;
643             scores[checked_size(position, "position")] = score;
644             max_score = std::max(max_score, score);
645         }
646
647         float probability_sum = 0.0F;
648         for (std::int32_t position = 0; position <= model_position; ++position) {
649             const std::size_t index = checked_size(position, "position");
650             const float unnormalized = std::exp(scores[index] - max_score);
651             probabilities[index] = unnormalized;
652             probability_sum += unnormalized;
653         }
654         if (probability_sum <= 0.0F) {
655             throw std::runtime_error("attention probability sum is invalid");
656         }
657         for (std::int32_t position = 0; position <= model_position; ++position) {
658             const float probability =
659                 probabilities[checked_size(position, "position")] / probability_sum;
660             const std::span<const float> value = cache.value(layer_id, position, kv_head);
661             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662                 output[query_base + checked_size(dim, "dim")] +=
663                     probability * value[checked_size(dim, "dim")];
664             }
665         }
666     }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670                                            std::span<const std::int32_t> token_ids) {
671     validate_config(model.config);
```

```
    if (token_ids.empty()) {
        throw std::runtime_error("reference decode needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_context, "max_context"
    )) {
        throw std::runtime_error("token_ids exceed max_context");
    }
    if (model.token_embedding.size() !=
        checked_size(model.config.vocab_size, "vocab_size") *
            checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("token embedding size mismatch");
    }
    if (model.final_norm_weight.size() != checked_size(model.config.hidden_size
    , "hidden_size")) {
        throw std::runtime_error("final norm weight size mismatch");
    }
    validate_linear(model.lm_head, "lm_head");

    ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers
    .size()));
    std::vector<float> hidden(checked_size(model.config.hidden_size, "
    hidden_size"), 0.0F);
    for (std::int32_t position = 0; position < static_cast<std::int32_t>(
    token_ids.size());
         ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "
    position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("token id out of vocabulary");
        }
        const std::span<const float> embedding =
            row_span(model.token_embedding, token_id, model.config.hidden_size)
    ;
        std::copy(embedding.begin(), embedding.end(), hidden.begin());
        for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
    model.layers.size());
             ++layer_id) {
            forward_layer(model.config,
                          model.layers[checked_size(layer_id, "layer_id")],
                          layer_id,
                          position,
                          cache,
                          hidden);
        }
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "
    hidden_size"), 0.0F);
    std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size
    "), 0.0F);
    rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)
    ;
```

```
    linear_f32(model.lm_head, normed, logits);

    ReferenceDecodeResult result;
    result.predicted_token = argmax(logits);
    result.logits = std::move(logits);
    return result;
}

ReferenceDecoderModel make_tiny_reference_decoder_model() {
    ReferenceDecoderModel model;
    model.config.hidden_size = 4;
    model.config.query_heads = 2;
    model.config.kv_heads = 1;
    model.config.head_dim = 2;
    model.config.intermediate_size = 6;
    model.config.vocab_size = 5;
    model.config.max_context = 8;
    model.config.rms_epsilon = 1.0e-5F;
    model.config.rope_base = 10000.0F;

    model.token_embedding = {
        0.20F, -0.10F, 0.05F, 0.30F,
        -0.40F, 0.10F, 0.25F, -0.20F,
        0.15F, 0.35F, -0.30F, 0.05F,
        0.50F, -0.25F, 0.10F, -0.15F,
        -0.05F, 0.20F, 0.40F, -0.35F,
    };

    DecoderLayerWeightsF32 layer;
    layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
    layer.wq = make_linear(4,
                           4,
                           {
                               0.20F, -0.10F, 0.05F, 0.30F,
                               -0.15F, 0.25F, 0.10F, -0.05F,
                               0.05F, 0.10F, -0.20F, 0.15F,
                               0.30F, -0.05F, 0.20F, 0.10F,
                           });
    layer.wk = make_linear(4,
                           2,
                           {
                               0.10F, 0.20F, -0.10F, 0.05F,
                               -0.05F, 0.15F, 0.25F, -0.20F,
                           });
    layer.wv = make_linear(4,
                           2,
                           {
                               0.25F, -0.05F, 0.10F, 0.15F,
                               -0.10F, 0.30F, 0.05F, 0.20F,
                           });
    layer.wo = make_linear(4,
                           4,
```

```
                            {
                                0.20F, 0.10F, -0.05F, 0.15F,
                                -0.10F, 0.25F, 0.20F, -0.05F,
                                0.05F, -0.20F, 0.30F, 0.10F,
                                0.15F, 0.05F, -0.10F, 0.25F,
                            });
    layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
    layer.w_gate = make_linear(4,
                               6,
                               {
                                   0.20F, -0.10F, 0.05F, 0.10F,
                                   -0.05F, 0.15F, 0.20F, -0.10F,
                                   0.10F, 0.05F, -0.15F, 0.25F,
                                   -0.20F, 0.10F, 0.15F, 0.05F,
                                   0.05F, -0.25F, 0.10F, 0.20F,
                                   0.15F, 0.05F, 0.05F, -0.15F,
                               });
    layer.w_up = make_linear(4,
                             6,
                             {
                                 0.10F, 0.20F, -0.05F, 0.05F,
                                 -0.15F, 0.05F, 0.25F, 0.10F,
                                 0.20F, -0.10F, 0.05F, 0.15F,
                                 0.05F, 0.10F, -0.20F, 0.25F,
                                 -0.05F, 0.15F, 0.10F, -0.10F,
                                 0.25F, -0.05F, 0.05F, 0.10F,
                             });
    layer.w_down = make_linear(6,
                               4,
                               {
                                   0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
                                   -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
                                   0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
                                   0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
                               });
    model.layers.push_back(std::move(layer));
    model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.lm_head = make_linear(4,
                                5,
                                {
                                    0.20F, -0.10F, 0.05F, 0.15F,
                                    -0.05F, 0.25F, 0.10F, -0.20F,
                                    0.15F, 0.05F, -0.25F, 0.10F,
                                    -0.10F, 0.20F, 0.15F, 0.05F,
                                    0.05F, -0.15F, 0.20F, 0.25F,
                                },
                                zeros(5));
    return model;
}

}  // namespace lcqi
```

Listing 16.20 src/gpt2_reference.cpp

```cpp
#include <lcqi/gpt2_reference.hpp>

#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <array>
#include <cmath>
#include <cstddef>
#include <fstream>
#include <limits>
#include <optional>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <unordered_set>
#include <utility>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
    -std::numeric_limits<float>::infinity();

class JsonCursor {
public:
    explicit JsonCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
```

```
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid JSON syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (ch == '\\') {
                result += this->parse_escape();
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated JSON string");
    }

    [[nodiscard]] std::int64_t parse_i64() {
        this->skip_ws();
        bool negative = false;
        if (this->consume('-')) {
            negative = true;
        }
        if (this->eof() || this->text_[this->position_] < '0' ||
            this->text_[this->position_] > '9') {
            throw std::runtime_error("expected JSON integer");
        }
        std::int64_t value = 0;
        while (!this->eof() && this->text_[this->position_] >= '0' &&
               this->text_[this->position_] <= '9') {
            const std::int64_t digit = this->text_[this->position_] - '0';
            if (value >
                (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
                throw std::runtime_error("JSON integer overflow");
            }
```

```
            value = value * 10 + digit;
            ++this->position_;
        }
        return negative ? -value : value;
    }

    [[nodiscard]] double parse_number() {
        this->skip_ws();
        const std::size_t begin = this->position_;
        if (!this->eof() && this->text_[this->position_] == '-') {
            ++this->position_;
        }
        while (!this->eof() && this->text_[this->position_] >= '0' &&
               this->text_[this->position_] <= '9') {
            ++this->position_;
        }
        if (!this->eof() && this->text_[this->position_] == '.') {
            ++this->position_;
            while (!this->eof() && this->text_[this->position_] >= '0' &&
                   this->text_[this->position_] <= '9') {
                ++this->position_;
            }
        }
        if (!this->eof() &&
            (this->text_[this->position_] == 'e' || this->text_[this->position_
    ↪ ] == 'E')) {
            ++this->position_;
            if (!this->eof() &&
                (this->text_[this->position_] == '+' || this->text_[this->
    ↪ position_] == '-')) {
                ++this->position_;
            }
            while (!this->eof() && this->text_[this->position_] >= '0' &&
                   this->text_[this->position_] <= '9') {
                ++this->position_;
            }
        }
        if (begin == this->position_) {
            throw std::runtime_error("expected JSON number");
        }
        return std::stod(std::string(this->text_.substr(begin, this->position_
    ↪ - begin)));
    }

    void skip_value() {
        this->skip_ws();
        if (this->eof()) {
            throw std::runtime_error("unexpected end of JSON");
        }
        const char ch = this->text_[this->position_];
        if (ch == '"') {
            static_cast<void>(this->parse_string());
```

```
            return;
        }
        if (ch == '{') {
            this->expect('{');
            if (this->consume('}')) {
                return;
            }
            while (true) {
                static_cast<void>(this->parse_string());
                this->expect(':');
                this->skip_value();
                if (this->consume('}')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == '[') {
            this->expect('[');
            if (this->consume(']')) {
                return;
            }
            while (true) {
                this->skip_value();
                if (this->consume(']')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == 't') {
            this->expect_literal("true");
            return;
        }
        if (ch == 'f') {
            this->expect_literal("false");
            return;
        }
        if (ch == 'n') {
            this->expect_literal("null");
            return;
        }
        static_cast<void>(this->parse_number());
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
```

```
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               (this->text_[this->position_] == ' ' ||
                this->text_[this->position_] == '\n' ||
                this->text_[this->position_] == '\r' ||
                this->text_[this->position_] == '\t')) {
            ++this->position_;
        }
    }

    void expect_literal(std::string_view literal) {
        for (const char expected : literal) {
            if (this->eof() || this->text_[this->position_] != expected) {
                throw std::runtime_error("invalid JSON literal");
            }
            ++this->position_;
        }
    }

    [[nodiscard]] std::string parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated JSON escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return std::string(1, escaped);
            case 'b':
                return "\b";
            case 'f':
                return "\f";
            case 'n':
                return "\n";
            case 'r':
                return "\r";
            case 't':
                return "\t";
            case 'u':
                return this->parse_unicode_escape();
            default:
                throw std::runtime_error("unsupported JSON escape");
        }
    }

```

```
    [[nodiscard]] std::string parse_unicode_escape() {
        std::uint32_t value = 0;
        for (std::int32_t i = 0; i < 4; ++i) {
            if (this->eof()) {
                throw std::runtime_error("unterminated JSON unicode escape");
            }
            const char ch = this->text_[this->position_++];
            value <<= 4U;
            if (ch >= '0' && ch <= '9') {
                value += static_cast<std::uint32_t>(ch - '0');
            } else if (ch >= 'a' && ch <= 'f') {
                value += static_cast<std::uint32_t>(10 + ch - 'a');
            } else if (ch >= 'A' && ch <= 'F') {
                value += static_cast<std::uint32_t>(10 + ch - 'A');
            } else {
                throw std::runtime_error("invalid JSON unicode escape");
            }
        }
        std::string encoded;
        if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(value));
        } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        } else {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        }
        return encoded;
    }
};

std::string read_text_file(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open text file: " + path.string());
    }
    std::ostringstream buffer;
    buffer << input.rdbuf();
    return buffer.str();
}

std::size_t checked_size(std::int64_t value, const char* name) {
```

```
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

std::string encode_codepoint(std::uint32_t value) {
    std::string encoded;
    if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(value));
    } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    }
    return encoded;
}

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const Gpt2Config& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.head_count, "head_count");
    require_positive(config.layer_count, "layer_count");
    require_positive(config.max_positions, "max_positions");
```

```
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.intermediate_size, "intermediate_size");
    if (config.hidden_size % config.head_count != 0) {
        throw std::runtime_error("hidden_size must be divisible by head_count");
    }
    if (config.layer_norm_epsilon <= 0.0F) {
        throw std::runtime_error("layer_norm_epsilon must be positive");
    }
}

std::int32_t head_dim(const Gpt2Config& config) {
    return config.hidden_size / config.head_count;
}

std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
    return checked_size(rows, "rows") * checked_size(columns, "columns");
}

void validate_vector_size(std::size_t actual, std::int32_t expected, const char* name) {
    if (actual != checked_size(expected, name)) {
        throw std::runtime_error(std::string(name) + " size mismatch");
    }
}

void validate_linear(const Gpt2LinearF32& linear, const char* name) {
    require_positive(linear.input_size, "linear input_size");
    require_positive(linear.output_size, "linear output_size");
    if (linear.weights.size() != matrix_size(linear.output_size, linear.input_size)) {
        throw std::runtime_error(std::string(name) + " weight size mismatch");
    }
    if (!linear.bias.empty() &&
        linear.bias.size() != checked_size(linear.output_size, "linear output_size")) {
        throw std::runtime_error(std::string(name) + " bias size mismatch");
    }
}

void validate_model(const Gpt2ReferenceModel& model) {
    validate_config(model.config);
    if (model.layers.size() != checked_size(model.config.layer_count, "layer_count")) {
        throw std::runtime_error("GPT-2 layer count mismatch");
    }
    if (model.token_embedding.size() !=
        matrix_size(model.config.vocab_size, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 token embedding size mismatch");
    }
    if (model.position_embedding.size() !=
        matrix_size(model.config.max_positions, model.config.hidden_size)) {
```

```
408         throw std::runtime_error("GPT-2 position embedding size mismatch");
409     }
410     validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size, "ln_f weight");
411     validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, "ln_f bias");
412     if (model.tie_lm_head_to_embedding) {
413         if (!model.lm_head_weight.empty()) {
414             throw std::runtime_error("tied GPT-2 lm_head should not carry separate weights");
415         }
416     } else if (model.lm_head_weight.size() !=
417                matrix_size(model.config.vocab_size, model.config.hidden_size)) {
418         throw std::runtime_error("GPT-2 lm_head size mismatch");
419     }
420     for (const Gpt2LayerWeightsF32& layer : model.layers) {
421         validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size, "ln_1 weight");
422         validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, "ln_1 bias");
423         validate_linear(layer.c_attn, "c_attn");
424         validate_linear(layer.c_proj, "c_proj");
425         validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size, "ln_2 weight");
426         validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, "ln_2 bias");
427         validate_linear(layer.c_fc, "c_fc");
428         validate_linear(layer.mlp_c_proj, "mlp_c_proj");
429     }
430 }
431
432 std::span<const float> row_span(std::span<const float> values,
433                                 std::int32_t row,
434                                 std::int32_t row_width) {
435     return values.subspan(checked_size(row, "row") * checked_size(row_width, "row_width"),
436                           checked_size(row_width, "row_width"));
437 }
438
439 void add_inplace(std::span<float> target, std::span<const float> source) {
440     if (target.size() != source.size()) {
441         throw std::runtime_error("GPT-2 residual add size mismatch");
442     }
443     for (std::size_t index = 0; index < target.size(); ++index) {
444         target[index] += source[index];
445     }
446 }
447
448 void layer_norm(std::span<const float> input,
449                 std::span<const float> weight,
450                 std::span<const float> bias,
```

```
                float epsilon,
                std::span<float> output) {
    if (input.empty() || input.size() != weight.size() || input.size() != bias.size() ||
        input.size() != output.size()) {
        throw std::runtime_error("GPT-2 LayerNorm size mismatch");
    }
    float mean = 0.0F;
    for (const float value : input) {
        mean += value;
    }
    mean /= static_cast<float>(input.size());

    float variance = 0.0F;
    for (const float value : input) {
        const float centered = value - mean;
        variance += centered * centered;
    }
    variance /= static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(variance + epsilon);

    for (std::size_t index = 0; index < input.size(); ++index) {
        output[index] = (input[index] - mean) * scale * weight[index] + bias[index];
    }
}

void linear_f32(const Gpt2LinearF32& linear,
                std::span<const float> input,
                std::span<float> output) {
    validate_linear(linear, "GPT-2 linear");
    if (input.size() != checked_size(linear.input_size, "linear input") ||
        output.size() != checked_size(linear.output_size, "linear output")) {
        throw std::runtime_error("GPT-2 linear input/output size mismatch");
    }
    for (std::int32_t out = 0; out < linear.output_size; ++out) {
        float sum = linear.bias.empty() ? 0.0F : linear.bias[checked_size(out, "out")];
        const std::size_t row_base = checked_size(out, "out") *
                                     checked_size(linear.input_size, "linear input");
        for (std::int32_t in = 0; in < linear.input_size; ++in) {
            sum += linear.weights[row_base + checked_size(in, "in")] *
                   input[checked_size(in, "in")];
        }
        output[checked_size(out, "out")] = sum;
    }
}

float gelu_new(float value) {
    const float cubic = value * value * value;
    return 0.5F * value *
```

```
            (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
                              (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
    }
    std::int32_t best_index = 0;
    float best_value = values.front();
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size↩
    ↪ ()); ++index) {
        const float value = values[checked_size(index, "argmax index")];
        if (value > best_value) {
            best_value = value;
            best_index = index;
        }
    }
    return best_index;
}

void project_qkv(const Gpt2Config& config,
                 const Gpt2LayerWeightsF32& layer,
                 std::span<const float> hidden,
                 std::span<float> q,
                 std::span<float> k,
                 std::span<float> v) {
    std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
                                          config.hidden_size),
                              0.0F);
    linear_f32(layer.c_attn, hidden, packed);
    const std::size_t width = checked_size(config.hidden_size, "hidden_size");
    std::copy_n(packed.begin(), config.hidden_size, q.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
                config.hidden_size,
                k.begin());
    std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
                config.hidden_size,
                v.begin());
}

void attention_cached_position(const Gpt2Config& config,
                               const Gpt2KvCache& cache,
                               std::int32_t layer_id,
                               std::int32_t model_position,
                               std::span<const float> query,
                               std::span<float> output) {
    const std::int32_t local_head_dim = head_dim(config);
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(↩
    ↪ local_head_dim));
    std::fill(output.begin(), output.end(), 0.0F);
    std::vector<float> scores(checked_size(model_position + 1, "model_position"↩
```

```
    ↪ ),
                              LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);

    for (std::int32_t head = 0; head < config.head_count; ++head) {
        float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
        for (std::int32_t past = 0; past <= model_position; ++past) {
            const std::span<const float> key = cache.key(layer_id, past, head);
            float dot = 0.0F;
            for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                const std::size_t query_index =
                    matrix_size(head, local_head_dim) + checked_size(dim, "head↩
    ↪  dim");
                dot += query[query_index] * key[checked_size(dim, "head dim")];
            }
            const float score = dot * score_scale;
            scores[checked_size(past, "past")] = score;
            max_score = std::max(max_score, score);
        }

        float denominator = 0.0F;
        for (std::int32_t past = 0; past <= model_position; ++past) {
            const std::size_t index = checked_size(past, "past");
            scores[index] = std::exp(scores[index] - max_score);
            denominator += scores[index];
        }
        if (denominator <= 0.0F) {
            throw std::runtime_error("GPT-2 cached attention denominator is ↩
    ↪ invalid");
        }

        for (std::int32_t past = 0; past <= model_position; ++past) {
            const float probability = scores[checked_size(past, "past")] / ↩
    ↪ denominator;
            const std::span<const float> value = cache.value(layer_id, past, ↩
    ↪ head);
            for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                const std::size_t output_index =
                    matrix_size(head, local_head_dim) + checked_size(dim, "head↩
    ↪  dim");
                output[output_index] += probability * value[checked_size(dim, "↩
    ↪ head dim")];
            }
        }
    }
}

void attention_full_sequence(const Gpt2Config& config,
                             std::span<const float> q_by_pos,
                             std::span<const float> k_by_pos,
                             std::span<const float> v_by_pos,
                             std::int32_t sequence_length,
                             std::span<float> output_by_pos) {
```

```
    const std::int32_t local_head_dim = head_dim(config);
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(local_head_dim));
    std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
    std::vector<float> scores(checked_size(sequence_length, "sequence_length"),
                              LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        for (std::int32_t head = 0; head < config.head_count; ++head) {
            float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
            for (std::int32_t past = 0; past <= position; ++past) {
                float dot = 0.0F;
                for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                    const std::size_t q_index =
                        matrix_size(position, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    const std::size_t k_index =
                        matrix_size(past, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    dot += q_by_pos[q_index] * k_by_pos[k_index];
                }
                const float score = dot * score_scale;
                scores[checked_size(past, "past")] = score;
                max_score = std::max(max_score, score);
            }

            float denominator = 0.0F;
            for (std::int32_t past = 0; past <= position; ++past) {
                const std::size_t index = checked_size(past, "past");
                scores[index] = std::exp(scores[index] - max_score);
                denominator += scores[index];
            }
            if (denominator <= 0.0F) {
                throw std::runtime_error("GPT-2 attention denominator is invalid");
            }
            for (std::int32_t past = 0; past <= position; ++past) {
                const float probability = scores[checked_size(past, "past")] / denominator;
                for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                    const std::size_t output_index =
                        matrix_size(position, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    const std::size_t value_index =
                        matrix_size(past, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    output_by_pos[output_index] += probability * v_by_pos[value_index];
```

```
                }
            }
        }
    }
}

void forward_gpt2_layer(const Gpt2Config& config,
                        const Gpt2LayerWeightsF32& layer,
                        std::int32_t sequence_length,
                        std::vector<float>& hidden_by_pos) {
    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size), 0.0F);
    std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> mlp_fc(checked_size(config.intermediate_size, "intermediate_size"), 0.0F);
    std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"), 0.0F);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        layer_norm(hidden,
                   layer.ln_1_weight,
                   layer.ln_1_bias,
                   config.layer_norm_epsilon,
                   normed);
        std::span<float> q = std::span<float>(
            q_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> k = std::span<float>(
            k_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> v = std::span<float>(
            v_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        project_qkv(config, layer, normed, q, k, v);
    }

    attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, sequence_length, attention_by_pos);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
```

```
        const std::span<const float> attention = std::span<const float>(
            attention_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        linear_f32(layer.c_proj, attention, projected);
        add_inplace(hidden, projected);

        layer_norm(hidden,
                   layer.ln_2_weight,
                   layer.ln_2_bias,
                   config.layer_norm_epsilon,
                   normed);
        linear_f32(layer.c_fc, normed, mlp_fc);
        for (float& value : mlp_fc) {
            value = gelu_new(value);
        }
        linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
        add_inplace(hidden, mlp_out);
    }
}

void forward_gpt2_layer_cached(const Gpt2Config& config,
                               const Gpt2LayerWeightsF32& layer,
                               std::int32_t layer_id,
                               std::int32_t model_position,
                               Gpt2KvCache& cache,
                               std::vector<float>& hidden) {
    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> key(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> value(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> mlp_fc(checked_size(config.intermediate_size, "intermediate_size"), 0.0F);
    std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"), 0.0F);

    layer_norm(hidden,
               layer.ln_1_weight,
               layer.ln_1_bias,
               config.layer_norm_epsilon,
               normed);
    project_qkv(config, layer, normed, query, key, value);
    cache.append(layer_id, model_position, key, value);
    attention_cached_position(config, cache, layer_id, model_position, query,
```

```
    attention);

    linear_f32(layer.c_proj, attention, projected);
    add_inplace(hidden, projected);

    layer_norm(hidden,
               layer.ln_2_weight,
               layer.ln_2_bias,
               config.layer_norm_epsilon,
               normed);
    linear_f32(layer.c_fc, normed, mlp_fc);
    for (float& value_ref : mlp_fc) {
        value_ref = gelu_new(value_ref);
    }
    linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
    add_inplace(hidden, mlp_out);
}

void compute_logits(const Gpt2ReferenceModel& model,
                    std::span<const float> hidden,
                    std::span<float> logits) {
    if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
        throw std::runtime_error("GPT-2 logits size mismatch");
    }
    const std::span<const float> weight =
        model.tie_lm_head_to_embedding ? std::span<const float>(model.
    token_embedding)
                                       : std::span<const float>(model.
    lm_head_weight);
    for (std::int32_t token = 0; token < model.config.vocab_size; ++token) {
        float sum = 0.0F;
        const std::span<const float> row = row_span(weight, token, model.config
    .hidden_size);
        for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
            sum += row[checked_size(dim, "dim")] * hidden[checked_size(dim, "
    dim")];
        }
        logits[checked_size(token, "token")] = sum;
    }
}

std::vector<std::string> gpt2_bytes_to_unicode() {
    std::vector<std::int32_t> bytes;
    for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
         value <= LCQI_GPT2_ASCII_TILDE;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
         value <= LCQI_GPT2_LATIN_NOT_SIGN;
         ++value) {
        bytes.push_back(value);
```

```
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
         value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
         ++value) {
        bytes.push_back(value);
    }

    std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
    std::vector<std::int32_t> chars = bytes;
    std::int32_t private_offset = 0;
    for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
        if (!byte_set.contains(byte)) {
            bytes.push_back(byte);
            chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
            ++private_offset;
        }
    }

    std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
    for (std::size_t index = 0; index < bytes.size(); ++index) {
        mapping[checked_size(bytes[index], "byte unicode index")] =
            encode_codepoint(static_cast<std::uint32_t>(chars[index]));
    }
    return mapping;
}

std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
    const std::vector<std::string>& byte_encoder) {
    std::unordered_map<std::string, unsigned char> result;
    for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
        result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
    }
    return result;
}

std::string bytes_to_bpe_alphabet(std::string_view text) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode
    ↪ ();
    std::string result;
    for (const unsigned char byte : text) {
        result += BYTE_ENCODER[byte];
    }
    return result;
}

std::vector<std::string> utf8_symbols(std::string_view text) {
    std::vector<std::string> symbols;
    std::size_t offset = 0;
    while (offset < text.size()) {
        const unsigned char lead = static_cast<unsigned char>(text[offset]);
        std::size_t width = 1;
        if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
```

```
            width = 2;
        } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
            width = 3;
        } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
            width = 4;
        }
        if (offset + width > text.size()) {
            throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
        }
        symbols.emplace_back(text.substr(offset, width));
        offset += width;
    }
    return symbols;
}

std::string pair_key(std::string_view left, std::string_view right) {
    std::string key(left);
    key.push_back('\n');
    key += right;
    return key;
}

std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::string_view token) {
    std::vector<std::string> word = utf8_symbols(token);
    if (word.size() <= 1) {
        return word;
    }

    while (true) {
        std::optional<std::int32_t> best_rank;
        std::string best_left;
        std::string best_right;
        for (std::size_t index = 0; index + 1 < word.size(); ++index) {
            const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index], word[index + 1]));
            if (rank != tokenizer.bpe_ranks.end() &&
                (!best_rank.has_value() || rank->second < *best_rank)) {
                best_rank = rank->second;
                best_left = word[index];
                best_right = word[index + 1];
            }
        }
        if (!best_rank.has_value()) {
            return word;
        }

        std::vector<std::string> merged;
        std::size_t index = 0;
        while (index < word.size()) {
            if (index + 1 < word.size() && word[index] == best_left &&
                word[index + 1] == best_right) {
```

```
                merged.push_back(best_left + best_right);
                index += LCQI_GPT2_PAIR_TOKEN_COUNT;
            } else {
                merged.push_back(word[index]);
                ++index;
            }
        }
        word = std::move(merged);
        if (word.size() == 1) {
            return word;
        }
    }
}

bool is_ascii_letter(unsigned char value) {
    return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
}

bool is_ascii_digit(unsigned char value) {
    return value >= '0' && value <= '9';
}

bool is_ascii_space(unsigned char value) {
    return value == ' ' || value == '\t' || value == '\n' ||
           value == '\r' || value == '\f' || value == '\v';
}

bool is_gpt2_punctuation(unsigned char value) {
    return !is_ascii_letter(value) && !is_ascii_digit(value) &&
           !is_ascii_space(value);
}

std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
    const unsigned char lead = static_cast<unsigned char>(text[offset]);
    std::size_t width = 1;
    if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
        width = 2;
    } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
        width = 3;
    } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
        width = 4;
    }
    if (offset + width > text.size()) {
        throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
    }
    return offset + width;
}

bool starts_with_contraction(std::string_view text,
                             std::size_t offset,
                             std::string_view contraction) {
    if (offset + contraction.size() > text.size()) {
```

```
        return false;
    }
    for (std::size_t index = 0; index < contraction.size(); ++index) {
        char left = text[offset + index];
        const char right = contraction[index];
        if (left >= 'A' && left <= 'Z') {
            left = static_cast<char>(left - 'A' + 'a');
        }
        if (left != right) {
            return false;
        }
    }
    return true;
}

std::vector<std::string> gpt2_pretokenize(std::string_view text) {
    std::vector<std::string> pieces;
    std::size_t offset = 0;
    const std::array<std::string_view, 7> contractions{
        "'s", "'t", "'re", "'ve", "'m", "'ll", "'d",
    };
    while (offset < text.size()) {
        bool emitted_contraction = false;
        for (const std::string_view contraction : contractions) {
            if (starts_with_contraction(text, offset, contraction)) {
                pieces.emplace_back(text.substr(offset, contraction.size()));
                offset += contraction.size();
                emitted_contraction = true;
                break;
            }
        }
        if (emitted_contraction) {
            continue;
        }

        const std::size_t begin = offset;
        bool has_prefix_space = false;
        if (text[offset] == ' ' && offset + 1 < text.size() &&
            !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
            has_prefix_space = true;
            ++offset;
        }

        if (offset >= text.size()) {
            pieces.emplace_back(text.substr(begin, offset - begin));
            continue;
        }

        const unsigned char ch = static_cast<unsigned char>(text[offset]);
        if (is_ascii_letter(ch)) {
            while (offset < text.size() &&
                   is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
```

```
                ++offset;
            }
        } else if (is_ascii_digit(ch)) {
            while (offset < text.size() &&
                   is_ascii_digit(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) {
            while (offset < text.size() &&
                   is_gpt2_punctuation(static_cast<unsigned char>(text[offset])) &&
                   static_cast<unsigned char>(text[offset]) != LCQI_GPT2_ASCII_APOSTROPHE) {
                offset = consume_utf8_codepoint(text, offset);
            }
        } else if (is_ascii_space(ch)) {
            while (offset < text.size() &&
                   is_ascii_space(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else {
            offset = consume_utf8_codepoint(text, offset);
        }

        if (has_prefix_space || offset > begin) {
            pieces.emplace_back(text.substr(begin, offset - begin));
        } else {
            ++offset;
        }
    }
    return pieces;
}

std::int32_t parse_i32_field(JsonCursor& cursor) {
    const std::int64_t value = cursor.parse_i64();
    if (value < std::numeric_limits<std::int32_t>::min() ||
        value > std::numeric_limits<std::int32_t>::max()) {
        throw std::runtime_error("JSON int32 field out of range");
    }
    return static_cast<std::int32_t>(value);
}

std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view text) {
    JsonCursor cursor(text);
    std::unordered_map<std::string, std::int32_t> vocab;
    cursor.expect('{');
    if (cursor.consume('}')) {
        return vocab;
    }
    while (true) {
```

```
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        const std::int32_t id = parse_i32_field(cursor);
        vocab.emplace(key, id);
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after vocab JSON");
    }
    return vocab;
}

std::vector<std::string> build_id_to_token(
    const std::unordered_map<std::string, std::int32_t>& vocab) {
    std::int32_t max_id = -1;
    for (const auto& [token, id] : vocab) {
        if (id < 0) {
            throw std::runtime_error("GPT-2 vocab id cannot be negative");
        }
        max_id = std::max(max_id, id);
    }
    std::vector<std::string> result(checked_size(max_id + 1, "max vocab id"));
    for (const auto& [token, id] : vocab) {
        std::string& slot = result[checked_size(id, "vocab id")];
        if (!slot.empty()) {
            throw std::runtime_error("duplicate GPT-2 vocab id");
        }
        slot = token;
    }
    return result;
}

std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view
    ↪ text) {
    std::unordered_map<std::string, std::int32_t> ranks;
    std::istringstream input{std::string(text)};
    std::string line;
    std::int32_t rank = 0;
    while (std::getline(input, line)) {
        if (line.empty() || line[0] == '#') {
            continue;
        }
        std::istringstream line_stream(line);
        std::string left;
        std::string right;
        if (!(line_stream >> left >> right)) {
            throw std::runtime_error("invalid GPT-2 merges line");
        }
        ranks.emplace(pair_key(left, right), rank);
```

```
        ++rank;
    }
    return ranks;
}

void transpose_hf_conv1d(std::vector<float>& values,
                         std::int32_t input_size,
                         std::int32_t output_size) {
    if (values.size() != matrix_size(input_size, output_size)) {
        throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
    }
    std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
    for (std::int32_t input = 0; input < input_size; ++input) {
        for (std::int32_t output = 0; output < output_size; ++output) {
            transposed[matrix_size(output, input_size) + checked_size(input, "input")] =
                values[matrix_size(input, output_size) + checked_size(output, "output")];
        }
    }
    values = std::move(transposed);
}

std::vector<float> require_tensor(const SafeTensorFile& file,
                                  std::string_view name,
                                  std::span<const std::int64_t> expected_shape) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
    }
    if (entry->shape.size() != expected_shape.size()) {
        throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(name));
    }
    for (std::size_t index = 0; index < expected_shape.size(); ++index) {
        if (entry->shape[index] != expected_shape[index]) {
            throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::string(name));
        }
    }
    return read_safetensor_f32_tensor(file, name);
}

std::vector<float> require_first_tensor(const SafeTensorFile& file,
                                        std::span<const std::string_view> names,
                                        std::span<const std::int64_t> expected_shape) {
    for (const std::string_view name : names) {
        if (file.manifest.find_tensor(name) != nullptr) {
            return require_tensor(file, name, expected_shape);
```

```
        }
    }
    throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front↩
    ↪ ()));
}

Gpt2LinearF32 make_linear(std::int32_t input_size,
                          std::int32_t output_size,
                          std::vector<float> weights,
                          std::vector<float> bias = {}) {
    Gpt2LinearF32 linear;
    linear.input_size = input_size;
    linear.output_size = output_size;
    linear.weights = std::move(weights);
    linear.bias = std::move(bias);
    validate_linear(linear, "make GPT-2 linear");
    return linear;
}

std::vector<float> zeros(std::int32_t count) {
    return std::vector<float>(checked_size(count, "zero count"), 0.0F);
}

std::vector<float> ones(std::int32_t count) {
    return std::vector<float>(checked_size(count, "one count"), 1.0F);
}

std::string tensor_name(std::int32_t layer, std::string_view suffix) {
    return "transformer.h." + std::to_string(layer) + "." + std::string(suffix)↩
    ↪ ;
}

std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
    return "h." + std::to_string(layer) + "." + std::string(suffix);
}

std::array<std::string_view, 2> embedding_names(std::string_view bare) {
    if (bare == "wte.weight") {
        return {"transformer.wte.weight", "wte.weight"};
    }
    if (bare == "wpe.weight") {
        return {"transformer.wpe.weight", "wpe.weight"};
    }
    if (bare == "ln_f.weight") {
        return {"transformer.ln_f.weight", "ln_f.weight"};
    }
    if (bare == "ln_f.bias") {
        return {"transformer.ln_f.bias", "ln_f.bias"};
    }
    throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
}

```

```
std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view suffix) {
    return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
}

std::vector<float> require_layer_tensor(const SafeTensorFile& file,
                                        std::int32_t layer,
                                        std::string_view suffix,
                                        std::span<const std::int64_t> expected_shape) {
    const std::array<std::string, 2> names = layer_names(layer, suffix);
    const std::array<std::string_view, 2> views{names[0], names[1]};
    return require_first_tensor(file, views, expected_shape);
}

std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& model_directory) {
    const std::array<const char*, 2> candidates{
        "model.safetensors",
        "pytorch_model.safetensors",
    };
    for (const char* candidate : candidates) {
        const std::filesystem::path path = model_directory / candidate;
        if (std::filesystem::exists(path)) {
            return path;
        }
    }
    throw std::runtime_error("cannot find model.safetensors in GPT-2 directory");
}

}  // namespace

Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
                                  const std::filesystem::path& merges_txt,
                                  std::int32_t bos_token_id,
                                  std::int32_t eos_token_id) {
    Gpt2Tokenizer tokenizer;
    tokenizer.bos_token_id = bos_token_id;
    tokenizer.eos_token_id = eos_token_id;
    tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
    tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
    tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
    if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
        throw std::runtime_error("GPT-2 tokenizer assets are empty");
    }
    return tokenizer;
}

std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
                                      std::string_view text) {
    std::vector<std::int32_t> ids;
```

```
    for (const std::string& piece : gpt2_pretokenize(text)) {
        const std::string byte_piece = bytes_to_bpe_alphabet(piece);
        const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, byte_piece);
        for (const std::string& token : bpe_tokens) {
            const auto found = tokenizer.vocab.find(token);
            if (found == tokenizer.vocab.end()) {
                throw std::runtime_error("GPT-2 BPE token is missing from vocab");
            }
            ids.push_back(found->second);
        }
    }
    return ids;
}

std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
                        std::span<const std::int32_t> token_ids) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
    static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
        gpt2_unicode_to_bytes(BYTE_ENCODER);

    std::string token_text;
    for (const std::int32_t token_id : token_ids) {
        if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.id_to_token.size())) {
            throw std::runtime_error("GPT-2 token id out of range");
        }
        token_text += tokenizer.id_to_token[checked_size(token_id, "token id")];
    }

    std::string result;
    const std::vector<std::string> symbols = utf8_symbols(token_text);
    for (const std::string& symbol : symbols) {
        const auto found = BYTE_DECODER.find(symbol);
        if (found == BYTE_DECODER.end()) {
            throw std::runtime_error("GPT-2 token cannot be decoded to byte");
        }
        result.push_back(static_cast<char>(found->second));
    }
    return result;
}

Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
    const std::string config_text = read_text_file(config_json);
    JsonCursor cursor(config_text);
    Gpt2Config config;
    config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
    config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;

```

```
    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("GPT-2 config is empty");
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        if (key == "n_embd") {
            config.hidden_size = parse_i32_field(cursor);
        } else if (key == "n_head") {
            config.head_count = parse_i32_field(cursor);
        } else if (key == "n_layer") {
            config.layer_count = parse_i32_field(cursor);
        } else if (key == "n_positions" || key == "n_ctx") {
            const std::int32_t value = parse_i32_field(cursor);
            config.max_positions = std::max(config.max_positions, value);
        } else if (key == "vocab_size") {
            config.vocab_size = parse_i32_field(cursor);
        } else if (key == "n_inner") {
            config.intermediate_size = parse_i32_field(cursor);
        } else if (key == "layer_norm_epsilon") {
            config.layer_norm_epsilon = static_cast<float>(cursor.parse_number
    ↪ ());
        } else if (key == "activation_function") {
            config.activation_function = cursor.parse_string();
        } else if (key == "bos_token_id") {
            config.bos_token_id = parse_i32_field(cursor);
        } else if (key == "eos_token_id") {
            config.eos_token_id = parse_i32_field(cursor);
        } else {
            cursor.skip_value();
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after GPT-2 config JSON");
    }
    if (config.intermediate_size == 0 && config.hidden_size > 0) {
        config.intermediate_size = config.hidden_size * 4;
    }
    validate_config(config);
    return config;
}

Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& 
    ↪ config_json,
                                                 const std::filesystem::path& 
    ↪ safetensors_path) {
    Gpt2ReferenceModel model;
```

```
    model.config = load_gpt2_config(config_json);
    const SafeTensorFile file = load_safetensors_file(safetensors_path);
    const Gpt2Config& config = model.config;
    const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.hidden_size};
    const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.hidden_size};
    const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};

    const std::array<std::string_view, 2> wte_names = embedding_names("wte.weight");
    const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.weight");
    model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
    model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape);
    model.layers.resize(checked_size(config.layer_count, "layer_count"));

    for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) {
        Gpt2LayerWeightsF32& layer = model.layers[checked_size(layer_id, "layer_id")];
        layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight", hidden_shape);
        layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", hidden_shape);
        layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight", hidden_shape);
        layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", hidden_shape);

        const std::array<std::int64_t, 2> c_attn_shape{
            config.hidden_size,
            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
        std::vector<float> c_attn_weight =
            require_layer_tensor(file, layer_id, "attn.c_attn.weight", c_attn_shape);
        transpose_hf_conv1d(c_attn_weight,
                            config.hidden_size,
                            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS);
        const std::array<std::int64_t, 1> c_attn_bias_shape{
            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
        layer.c_attn = make_linear(config.hidden_size,
                                   config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
                                   std::move(c_attn_weight),
                                   require_layer_tensor(file,
                                                        layer_id,
                                                        "attn.c_attn.bias",
                                                        c_attn_bias_shape));

```

```
        const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, config.hidden_size};
        std::vector<float> c_proj_weight =
            require_layer_tensor(file, layer_id, "attn.c_proj.weight", c_proj_shape);
        transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.hidden_size);
        layer.c_proj = make_linear(config.hidden_size,
                                   config.hidden_size,
                                   std::move(c_proj_weight),
                                   require_layer_tensor(file,
                                                        layer_id,
                                                        "attn.c_proj.bias",
                                                        hidden_shape));

        const std::array<std::int64_t, 2> c_fc_shape{
            config.hidden_size,
            config.intermediate_size};
        std::vector<float> c_fc_weight =
            require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape);
        transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.intermediate_size);
        const std::array<std::int64_t, 1> intermediate_shape{config.intermediate_size};
        layer.c_fc = make_linear(config.hidden_size,
                                 config.intermediate_size,
                                 std::move(c_fc_weight),
                                 require_layer_tensor(file,
                                                      layer_id,
                                                      "mlp.c_fc.bias",
                                                      intermediate_shape));

        const std::array<std::int64_t, 2> mlp_proj_shape{
            config.intermediate_size,
            config.hidden_size};
        std::vector<float> mlp_proj_weight =
            require_layer_tensor(file, layer_id, "mlp.c_proj.weight", mlp_proj_shape);
        transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.hidden_size);
        layer.mlp_c_proj = make_linear(config.intermediate_size,
                                       config.hidden_size,
                                       std::move(mlp_proj_weight),
                                       require_layer_tensor(file,
                                                            layer_id,
                                                            "mlp.c_proj.bias",
                                                            hidden_shape));
    }

    const std::array<std::string_view, 2> ln_weight_names = embedding_names("ln_f.weight");
```

```
    const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f.bias");
    model.final_ln_weight = require_first_tensor(file, ln_weight_names, hidden_shape);
    model.final_ln_bias = require_first_tensor(file, ln_bias_names, hidden_shape);
    if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
        model.tie_lm_head_to_embedding = false;
        model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape);
    } else {
        model.tie_lm_head_to_embedding = true;
    }
    validate_model(model);
    return model;
}

Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& model_directory) {
    return load_gpt2_reference_model(model_directory / "config.json",
                                     find_gpt2_weight_file(model_directory));
}

Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
    validate_config(this->config_);
    const std::size_t element_count =
        checked_size(this->config_.layer_count, "layer_count") *
        checked_size(this->config_.max_positions, "max_positions") *
        checked_size(this->config_.head_count, "head_count") *
        checked_size(head_dim(this->config_), "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->config_.layer_count, "layer_count") *
            checked_size(this->config_.max_positions, "max_positions"),
        0);
}

void Gpt2KvCache::append(std::int32_t layer_id,
                         std::int32_t model_position,
                         std::span<const float> key,
                         std::span<const float> value) {
    const std::size_t hidden_size = checked_size(this->config_.hidden_size, "hidden_size");
    if (key.size() != hidden_size || value.size() != hidden_size) {
        throw std::runtime_error("GPT-2 KV key/value size mismatch");
    }
    this->validate_address(layer_id, model_position, 0);
    const std::int32_t local_head_dim = head_dim(this->config_);
    for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
        const std::size_t source =
            checked_size(head, "head") * checked_size(local_head_dim, "head_dim
```

```
        ↪ ");
1444        const std::size_t target = this->base_offset(layer_id, model_position, ↩
        ↪ head);
1445        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
1446                    local_head_dim,
1447                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
1448        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
1449                    local_head_dim,
1450                    this->values_.begin() + static_cast<std::ptrdiff_t>(target)↩
        ↪ );
1451    }
1452    const std::size_t written_index =
1453        checked_size(layer_id, "layer_id") *
1454            checked_size(this->config_.max_positions, "max_positions") +
1455        checked_size(model_position, "model_position");
1456    this->written_[written_index] = 1;
1457    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
1458 }
1459
1460 std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
1461                                         std::int32_t model_position,
1462                                         std::int32_t head) const {
1463    this->validate_address(layer_id, model_position, head);
1464    this->validate_written(layer_id, model_position);
1465    const std::size_t offset = this->base_offset(layer_id, model_position, head↩
        ↪ );
1466    return std::span<const float>(this->keys_.data() + offset,
1467                                  checked_size(head_dim(this->config_), "↩
        ↪ head_dim"));
1468 }
1469
1470 std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
1471                                           std::int32_t model_position,
1472                                           std::int32_t head) const {
1473    this->validate_address(layer_id, model_position, head);
1474    this->validate_written(layer_id, model_position);
1475    const std::size_t offset = this->base_offset(layer_id, model_position, head↩
        ↪ );
1476    return std::span<const float>(this->values_.data() + offset,
1477                                  checked_size(head_dim(this->config_), "↩
        ↪ head_dim"));
1478 }
1479
1480 std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
1481    return this->filled_tokens_;
1482 }
1483
1484 std::size_t Gpt2KvCache::byte_size() const noexcept {
1485    return (this->keys_.size() + this->values_.size()) * sizeof(float) +
1486           this->written_.size() * sizeof(std::uint8_t);
1487 }
1488
```

```
std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
                                     std::int32_t model_position,
                                     std::int32_t head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_positions, "max_positions")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.head_count, "head_count") +
            checked_size(head, "head")) *
           checked_size(head_dim(this->config_), "head_dim");
}

void Gpt2KvCache::validate_address(std::int32_t layer_id,
                                   std::int32_t model_position,
                                   std::int32_t head) const {
    if (layer_id < 0 || layer_id >= this->config_.layer_count) {
        throw std::runtime_error("GPT-2 KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_positions) {
        throw std::runtime_error("GPT-2 KV model_position out of range");
    }
    if (head < 0 || head >= this->config_.head_count) {
        throw std::runtime_error("GPT-2 KV head out of range");
    }
}

void Gpt2KvCache::validate_written(std::int32_t layer_id,
                                   std::int32_t model_position) const {
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") *
            checked_size(this->config_.max_positions, "max_positions") +
        checked_size(model_position, "model_position");
    if (written_index >= this->written_.size() || this->written_[written_index]
    ↪  == 0) {
        throw std::runtime_error("GPT-2 KV slot has not been written");
    }
}

Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
                                   std::span<const std::int32_t> token_ids) {
    validate_model(model);
    if (token_ids.empty()) {
        throw std::runtime_error("GPT-2 forward needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_positions, "
    ↪ max_positions")) {
        throw std::runtime_error("GPT-2 token_ids exceed max_positions");
    }

    const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.
    ↪ size());
    std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.
    ↪ hidden_size), 0.0F);
```

```
    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("GPT-2 token id out of range");
        }
        const std::span<const float> token =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        const std::span<const float> position_embedding =
            row_span(model.position_embedding, position, model.config.hidden_size);
        std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, model.config.hidden_size),
            checked_size(model.config.hidden_size, "hidden_size"));
        for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
            hidden[checked_size(dim, "dim")] =
                token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
        }
    }

    for (const Gpt2LayerWeightsF32& layer : model.layers) {
        forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos);
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    const std::span<const float> final_hidden = std::span<const float>(
        hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.hidden_size),
        checked_size(model.config.hidden_size, "hidden_size"));
    layer_norm(final_hidden,
               model.final_ln_weight,
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);

    Gpt2ForwardResult result;
    result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    compute_logits(model, normed, result.logits);
    result.predicted_token = argmax(result.logits);
    return result;
}

Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
                                          Gpt2KvCache& cache,
                                          std::int32_t token_id) {
    validate_model(model);
```

```
    if (token_id < 0 || token_id >= model.config.vocab_size) {
        throw std::runtime_error("GPT-2 token id out of range");
    }
    const std::int32_t model_position = cache.filled_tokens();
    if (model_position >= model.config.max_positions) {
        throw std::runtime_error("GPT-2 cached forward would exceed max_positions");
    }

    std::vector<float> hidden(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    const std::span<const float> token =
        row_span(model.token_embedding, token_id, model.config.hidden_size);
    const std::span<const float> position_embedding =
        row_span(model.position_embedding, model_position, model.config.hidden_size);
    for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
        hidden[checked_size(dim, "dim")] =
            token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
    }

    for (std::int32_t layer_id = 0;
         layer_id < static_cast<std::int32_t>(model.layers.size());
         ++layer_id) {
        forward_gpt2_layer_cached(model.config,
                                  model.layers[checked_size(layer_id, "layer_id")],
                                  layer_id,
                                  model_position,
                                  cache,
                                  hidden);
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    layer_norm(hidden,
               model.final_ln_weight,
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);

    Gpt2ForwardResult result;
    result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    compute_logits(model, normed, result.logits);
    result.predicted_token = argmax(result.logits);
    return result;
}

std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
                                               std::span<const std::int32_t>
```

```
    prompt_token_ids,
                                                std::int32_t max_new_tokens) {
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    return tokens;
}

std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens) {
    if (prompt_token_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    if (prompt_token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 prompt exceeds max_positions");
    }

    Gpt2KvCache cache(model.config);
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }
    Gpt2ForwardResult result;
    for (const std::int32_t token_id : prompt_token_ids) {
        result = run_gpt2_forward_cached(model, cache, token_id);
    }
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
```

```
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            result = run_gpt2_forward_cached(model, cache, result.predicted_token);
        }
    }
    return tokens;
}

Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
    Gpt2ReferenceModel model;
    model.config.hidden_size = 4;
    model.config.head_count = 2;
    model.config.layer_count = 1;
    model.config.max_positions = 8;
    model.config.vocab_size = 6;
    model.config.intermediate_size = 8;
    model.config.bos_token_id = 0;
    model.config.eos_token_id = 5;
    model.config.layer_norm_epsilon = 1.0e-5F;
    model.config.activation_function = "gelu_new";

    model.token_embedding = {
        0.20F, -0.10F, 0.00F, 0.10F,
        -0.10F, 0.30F, 0.20F, -0.20F,
        0.05F, 0.10F, -0.30F, 0.25F,
        0.40F, -0.20F, 0.15F, 0.05F,
        -0.30F, 0.05F, 0.35F, -0.15F,
        0.10F, 0.20F, -0.10F, 0.30F,
    };
    model.position_embedding = {
        0.01F, 0.02F, 0.03F, 0.04F,
        -0.02F, 0.01F, 0.00F, 0.03F,
        0.03F, -0.01F, 0.02F, 0.00F,
        0.00F, 0.02F, -0.02F, 0.01F,
        0.02F, 0.00F, 0.01F, -0.01F,
        -0.01F, 0.03F, 0.00F, 0.02F,
        0.01F, -0.02F, 0.03F, 0.00F,
        0.00F, 0.01F, -0.01F, 0.02F,
    };

    Gpt2LayerWeightsF32 layer;
    layer.ln_1_weight = ones(4);
    layer.ln_1_bias = zeros(4);
    layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
```

```
    layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
    layer.c_attn = make_linear(
        4,
        12,
        {
            0.10F, -0.20F, 0.05F, 0.15F,
            -0.05F, 0.12F, 0.20F, -0.10F,
            0.18F, 0.02F, -0.08F, 0.11F,
            -0.12F, 0.06F, 0.14F, 0.04F,
            0.07F, 0.16F, -0.09F, 0.03F,
            -0.15F, 0.05F, 0.13F, 0.10F,
            0.04F, -0.11F, 0.19F, -0.02F,
            0.09F, 0.08F, -0.06F, 0.17F,
            0.20F, -0.04F, 0.01F, 0.06F,
            -0.08F, 0.15F, 0.07F, -0.03F,
            0.05F, 0.10F, -0.14F, 0.12F,
            0.11F, -0.07F, 0.16F, 0.02F,
        },
        {
            0.01F, -0.02F, 0.00F, 0.03F,
            -0.01F, 0.02F, 0.01F, -0.02F,
            0.00F, 0.01F, -0.01F, 0.02F,
        });
    layer.c_proj = make_linear(
        4,
        4,
        {
            0.10F, 0.05F, -0.08F, 0.12F,
            -0.04F, 0.11F, 0.09F, -0.02F,
            0.07F, -0.10F, 0.13F, 0.05F,
            0.02F, 0.14F, -0.06F, 0.08F,
        },
        {0.01F, 0.00F, -0.01F, 0.02F});
    layer.c_fc = make_linear(
        4,
        8,
        {
            0.20F, -0.10F, 0.05F, 0.03F,
            -0.05F, 0.14F, 0.12F, -0.08F,
            0.09F, 0.07F, -0.11F, 0.16F,
            -0.12F, 0.05F, 0.18F, 0.02F,
            0.04F, -0.09F, 0.13F, 0.10F,
            0.11F, 0.03F, -0.07F, 0.15F,
            -0.02F, 0.17F, 0.06F, -0.04F,
            0.08F, -0.06F, 0.10F, 0.12F,
        },
        {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
    layer.mlp_c_proj = make_linear(
        8,
        4,
        {
            0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
```

```
            -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
            0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
            0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
        },
        {0.00F, 0.01F, -0.01F, 0.02F});
    model.layers.push_back(std::move(layer));
    model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
    model.tie_lm_head_to_embedding = true;
    validate_model(model);
    return model;
}

}  // namespace lcqi
```

Listing 16.21 src/safetensors.cpp

```
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <cctype>
#include <cstddef>
#include <cmath>
#include <cstring>
#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U;
constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
constexpr std::uint64_t LCQI_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
```

```
constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;

class HeaderCursor {
public:
    explicit HeaderCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid safetensors header syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) {
    ↪ {
                throw std::runtime_error("control character in safetensors
    ↪ string");
            }
            if (ch == '\\') {
                result.push_back(this->parse_escape());
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated safetensors string");
    }

```

```
    [[nodiscard]] std::uint64_t parse_u64() {
        this->skip_ws();
        if (this->eof() ||
            !std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            throw std::runtime_error("expected unsigned integer in safetensors header");
        }
        std::uint64_t value = 0;
        while (!this->eof() &&
               std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            const std::uint64_t digit =
                static_cast<std::uint64_t>(this->text_[this->position_] - '0');
            if (value >
                (std::numeric_limits<std::uint64_t>::max() - digit) / LCQI_DECIMAL_BASE) {
                throw std::runtime_error("safetensors integer overflow");
            }
            value = value * LCQI_DECIMAL_BASE + digit;
            ++this->position_;
        }
        return value;
    }

    [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
        std::vector<std::int64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            const std::uint64_t raw = this->parse_u64();
            if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::int64_t>::max())) {
                throw std::runtime_error("safetensors shape value is too large");
            }
            values.push_back(static_cast<std::int64_t>(raw));
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
        std::vector<std::uint64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
```

```
        while (true) {
            values.push_back(this->parse_u64());
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               std::isspace(static_cast<unsigned char>(this->text_[this->position_]))) {
            ++this->position_;
        }
    }

    [[nodiscard]] char parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated safetensors escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return escaped;
            case 'b':
                return '\b';
            case 'f':
                return '\f';
            case 'n':
                return '\n';
            case 'r':
                return '\r';
            case 't':
                return '\t';
            default:
                throw std::runtime_error("unsupported safetensors string escape
```

```
        ↪ ");
182         }
183     }
184 };
185
186 std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
187     if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
188         throw std::runtime_error("safetensors file is too small");
189     }
190     std::uint64_t value = 0;
191     for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
192         value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
193     }
194     return value;
195 }
196
197 std::string read_file_bytes(const std::filesystem::path& path) {
198     std::ifstream input(path, std::ios::binary);
199     if (!input) {
200         throw std::runtime_error("cannot open safetensors file: " + path.string↩
        ↪ ());
201     }
202     input.seekg(0, std::ios::end);
203     const std::streamoff size = input.tellg();
204     if (size < 0) {
205         throw std::runtime_error("cannot determine safetensors file size");
206     }
207     input.seekg(0, std::ios::beg);
208     std::string bytes(static_cast<std::size_t>(size), '\0');
209     if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize↩
        ↪ >(bytes.size()))) {
210         throw std::runtime_error("cannot read safetensors file");
211     }
212     return bytes;
213 }
214
215 std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
216     std::ifstream input(path, std::ios::binary);
217     if (!input) {
218         throw std::runtime_error("cannot open safetensors file: " + path.string↩
        ↪ ());
219     }
220     input.seekg(0, std::ios::end);
221     const std::streamoff size = input.tellg();
222     if (size < 0) {
223         throw std::runtime_error("cannot determine safetensors file size");
224     }
225     input.seekg(0, std::ios::beg);
226     std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
227     if (!bytes.empty() &&
228         !input.read(reinterpret_cast<char*>(bytes.data()),
229                     static_cast<std::streamsize>(bytes.size()))) {
```

```
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

void parse_metadata_value(HeaderCursor& cursor,
                          SafeTensorManifest& manifest,
                          const std::string& key) {
    if (key == "__metadata__") {
        cursor.expect('{');
        if (cursor.consume('}')) {
            return;
        }
        while (true) {
            const std::string metadata_key = cursor.parse_string();
            cursor.expect(':');
            const std::string metadata_value = cursor.parse_string();
            manifest.metadata.emplace_back(metadata_key, metadata_value);
            if (cursor.consume('}')) {
                return;
            }
            cursor.expect(',');
        }
    }
    throw std::runtime_error("unexpected safetensors metadata value");
}

SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& name) {
    SafeTensorEntry entry;
    entry.name = name;
    bool saw_dtype = false;
    bool saw_shape = false;
    bool saw_offsets = false;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("empty safetensors tensor entry");
    }
    while (true) {
        const std::string field = cursor.parse_string();
        cursor.expect(':');
        if (field == "dtype") {
            entry.dtype = cursor.parse_string();
            saw_dtype = true;
        } else if (field == "shape") {
            entry.shape = cursor.parse_i64_array();
            saw_shape = true;
        } else if (field == "data_offsets") {
            const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
            if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
```

```
                throw std::runtime_error("safetensors data_offsets must contain two values");
            }
            entry.data_begin = offsets[0];
            entry.data_end = offsets[1];
            saw_offsets = true;
        } else {
            throw std::runtime_error("unsupported safetensors tensor field: " + field);
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }

    if (!saw_dtype || !saw_shape || !saw_offsets) {
        throw std::runtime_error("safetensors tensor entry is missing required fields");
    }
    if (entry.data_end < entry.data_begin) {
        throw std::runtime_error("safetensors tensor offset range is invalid");
    }
    return entry;
}

void parse_header(std::string_view header, SafeTensorManifest& manifest) {
    HeaderCursor cursor(header);
    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("safetensors header is empty");
    }

    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        if (key == "__metadata__") {
            parse_metadata_value(cursor, manifest, key);
        } else {
            manifest.tensors.push_back(parse_tensor_entry(cursor, key));
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }

    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after safetensors header");
    }
}

```

```
std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);

void validate_manifest(const SafeTensorManifest& manifest,
                       std::uint64_t file_size) {
    const std::uint64_t payload_size = file_size - manifest.data_start_offset;
    for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
        const SafeTensorEntry& left = manifest.tensors[i];
        if (left.name.empty() || left.dtype.empty()) {
            throw std::runtime_error("safetensors tensor has empty name or dtype");
        }
        if (left.data_end > payload_size) {
            throw std::runtime_error("safetensors tensor data_offsets exceed payload size");
        }
        const std::uint64_t expected_bytes = expected_tensor_bytes(left);
        if (left.byte_size() != expected_bytes) {
            throw std::runtime_error("safetensors tensor byte size does not match dtype and shape");
        }
        for (const std::int64_t dim : left.shape) {
            if (dim < 0) {
                throw std::runtime_error("safetensors shape dimension is negative");
            }
        }
        for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
            const SafeTensorEntry& right = manifest.tensors[j];
            if (left.name == right.name) {
                throw std::runtime_error("duplicate safetensors tensor name");
            }
            const bool overlaps =
                left.data_begin < right.data_end && right.data_begin < left.data_end;
            if (overlaps) {
                throw std::runtime_error("overlapping safetensors tensor data range");
            }
        }
    }
}

std::uint64_t dtype_byte_size(std::string_view dtype) {
    if (dtype == "F64") {
        return LCQI_DTYPE_BYTES_F64;
    }
    if (dtype == "F32") {
        return LCQI_DTYPE_BYTES_F32;
    }
    if (dtype == "F16" || dtype == "BF16") {
        return LCQI_DTYPE_BYTES_F16;
    }
```

```
    if (dtype == "I64" || dtype == "U64") {
        return LCQI_DTYPE_BYTES_I64;
    }
    if (dtype == "I32" || dtype == "U32") {
        return LCQI_DTYPE_BYTES_I32;
    }
    if (dtype == "I16" || dtype == "U16") {
        return LCQI_DTYPE_BYTES_I16;
    }
    if (dtype == "I8") {
        return LCQI_DTYPE_BYTES_I8;
    }
    if (dtype == "U8" || dtype == "BOOL") {
        return LCQI_DTYPE_BYTES_U8;
    }
    throw std::runtime_error("unsupported safetensors dtype: " + std::string(dtype));
}

std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
    std::uint64_t element_count = 1;
    for (const std::int64_t dim : entry.shape) {
        if (dim < 0) {
            throw std::runtime_error("safetensors shape dimension is negative");
        }
        const std::uint64_t extent = static_cast<std::uint64_t>(dim);
        if (extent != 0 &&
            element_count > std::numeric_limits<std::uint64_t>::max() / extent) {
            throw std::runtime_error("safetensors shape element count overflow");
        }
        element_count *= extent;
    }
    const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
    if (dtype_bytes != 0 &&
        element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes) {
        throw std::runtime_error("safetensors tensor byte size overflow");
    }
    return element_count * dtype_bytes;
}

SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> bytes) {
    const std::uint64_t header_size = read_le_u64(bytes);
    if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
        throw std::runtime_error("safetensors header size is invalid");
    }
    const std::uint64_t data_start_offset =
        static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size
```

```
    ↪ ;
    if (data_start_offset > bytes.size()) {
        throw std::runtime_error("safetensors header extends beyond file size")↩
    ↪ ;
    }

    SafeTensorManifest manifest;
    manifest.header_size = header_size;
    manifest.data_start_offset = data_start_offset;
    const char* header_begin =
        reinterpret_cast<const char*>(bytes.data() + ↩
    ↪ LCQI_SAFETENSORS_PREFIX_BYTES);
    const std::string_view header(header_begin, static_cast<std::size_t>(↩
    ↪ header_size));
    parse_header(header, manifest);
    validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
    return manifest;
}

std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t ↩
    ↪ offset) {
    return static_cast<std::uint16_t>(
        static_cast<std::uint16_t>(bytes[offset]) |
        (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
}

std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t ↩
    ↪ offset) {
    std::uint32_t value = 0;
    for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
        value |= static_cast<std::uint32_t>(bytes[offset + i]) << (↩
    ↪ LCQI_BYTE_BITS * i);
    }
    return value;
}

float bits_to_float(std::uint32_t bits) {
    float value = 0.0F;
    std::memcpy(&value, &bits, sizeof(value));
    return value;
}

float f16_to_f32(std::uint16_t bits) {
    const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_SIGN_MASK)
                               << LCQI_F16_TO_F32_SIGN_SHIFT;
    const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_EXPONENT_MASK;
    const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_MANTISSA_MASK;

    if (exponent == 0U) {
```

```
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467                       static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS))↩
    ↪ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480          LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492                                                   const SafeTensorEntry& entry)↩
    ↪  {
493     const std::uint64_t begin = file.manifest.data_start_offset + entry.↩
    ↪ data_begin;
494     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
495     if (begin > end || end > file.bytes.size()) {
496         throw std::runtime_error("safetensors tensor byte range is invalid");
497     }
498     return std::span<const std::uint8_t>(
499         file.bytes.data() + static_cast<std::size_t>(begin),
500         static_cast<std::size_t>(end - begin));
501 }
502
503 }  // namespace
504
505 std::uint64_t SafeTensorEntry::byte_size() const {
506     return this->data_end - this->data_begin;
507 }
508
509 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ↩
    ↪ const {
510     const auto found = std::find_if(
```

```
511         this->tensors.begin(),
512         this->tensors.end(),
513         [name](const SafeTensorEntry& entry) {
514             return entry.name == name;
515         });
516     if (found == this->tensors.end()) {
517         return nullptr;
518     }
519     return &(*found);
520 }
521
522 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path)↩
    ↪  {
523     const std::string bytes = read_file_bytes(path);
524     return parse_manifest_from_bytes(
525         std::span<const std::uint8_t>(
526             reinterpret_cast<const std::uint8_t*>(bytes.data()),
527             bytes.size()));
528 }
529
530 SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
531     SafeTensorFile file;
532     file.path = path;
533     file.bytes = read_file_u8(path);
534     file.manifest = parse_manifest_from_bytes(file.bytes);
535     return file;
536 }
537
538 std::span<const std::uint8_t> read_safetensor_tensor_bytes(
539     const SafeTensorFile& file,
540     std::string_view name) {
541     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
542     if (entry == nullptr) {
543         throw std::runtime_error("missing safetensors tensor: " + std::string(↩
    ↪ name));
544     }
545     return tensor_payload_span(file, *entry);
546 }
547
548 std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,
549                                               std::string_view name) {
550     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
551     if (entry == nullptr) {
552         throw std::runtime_error("missing safetensors tensor: " + std::string(↩
    ↪ name));
553     }
554     const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *↩
    ↪ entry);
555     std::vector<float> values;
556     if (entry->dtype == "F32") {
557         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
558         for (std::size_t index = 0; index < values.size(); ++index) {
```

```
            values[index] = bits_to_float(
                read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
        }
        return values;
    }
    if (entry->dtype == "F16") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = f16_to_f32(
                read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
        }
        return values;
    }
    if (entry->dtype == "BF16") {
        values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
        for (std::size_t index = 0; index < values.size(); ++index) {
            values[index] = bf16_to_f32(
                read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
        }
        return values;
    }
    throw std::runtime_error("safetensors tensor is not float-compatible: " + entry->name);
}

}  // namespace lcqi
```

Listing 16.22 src/tokenizer.cpp

```
#include <lcqi/tokenizer.hpp>

#include <cstddef>
#include <fstream>
#include <stdexcept>
#include <string>
#include <string_view>

namespace lcqi {
namespace {

constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 14695981039346656037ULL;
constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of tokenizer file");
    }
    return key;
}

```

```cpp
void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected tokenizer key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer int32 value for key '" + key + "'");
    }
    return value;
}

std::string read_string(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::string value;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer string value for key '" + key + "'");
    }
    return value;
}

std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view text) {
    for (const TokenEntry& entry : tokenizer.vocab) {
        if (entry.text == text) {
            return entry.id;
        }
    }
    return LCQI_INVALID_TOKEN_ID;
}

void validate_tokenizer(const TokenizerModel& tokenizer) {
    if (tokenizer.vocab.empty()) {
        throw std::runtime_error("tokenizer vocab is empty");
    }
    for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
        const TokenEntry& left = tokenizer.vocab[i];
        if (left.id < 0) {
            throw std::runtime_error("tokenizer token id must be non-negative");
        }
        for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
            const TokenEntry& right = tokenizer.vocab[j];
            if (left.id == right.id) {
                throw std::runtime_error("duplicate tokenizer token id");
```

```
            }
            if (left.text == right.text) {
                throw std::runtime_error("duplicate tokenizer token text");
            }
        }
    }
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
        throw std::runtime_error("tokenizer BOS id does not match vocab");
    }
    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
        throw std::runtime_error("tokenizer EOS id does not match vocab");
    }
}

std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
    hash ^= static_cast<std::uint64_t>(value);
    hash *= LCQI_FNV_PRIME;
    return hash;
}

std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
    for (const unsigned char value : text) {
        hash = hash_byte(hash, value);
    }
    return hash;
}

std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
    const std::uint32_t bits = static_cast<std::uint32_t>(value);
    for (std::int32_t shift = 0; shift < 32; shift += 8) {
        hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0↩
    ↪ xFFU));
    }
    return hash;
}

}  // namespace

TokenizerModel load_tokenizer(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open tokenizer file: " + path.string()↩
    ↪ );
    }

    expect_key(input, "LCQI_TOKENIZER_V1");

    TokenizerModel tokenizer;
    tokenizer.bos_id = read_i32(input, "bos_id");
    tokenizer.eos_id = read_i32(input, "eos_id");
```

```
    tokenizer.unk_id = read_i32(input, "unk_id");
    tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
    const std::int32_t vocab_size = read_i32(input, "vocab_size");
    if (vocab_size <= 0) {
        throw std::runtime_error("tokenizer vocab_size must be positive");
    }

    tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
    expect_key(input, "vocab");
    for (std::int32_t i = 0; i < vocab_size; ++i) {
        TokenEntry entry;
        if (!(input >> entry.id >> entry.text)) {
            throw std::runtime_error("invalid tokenizer vocab entry");
        }
        tokenizer.vocab.push_back(entry);
    }

    validate_tokenizer(tokenizer);
    return tokenizer;
}

std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
                                        std::string_view prompt) {
    std::vector<std::int32_t> ids;
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.bos_id);
    }

    std::size_t begin = 0;
    while (begin < prompt.size()) {
        while (begin < prompt.size() && prompt[begin] == ' ') {
            ++begin;
        }
        if (begin >= prompt.size()) {
            break;
        }
        std::size_t end = begin;
        while (end < prompt.size() && prompt[end] != ' ') {
            ++end;
        }
        const std::string_view token = prompt.substr(begin, end - begin);
        const std::int32_t token_id = lookup_token_id(tokenizer, token);
        if (token_id == LCQI_INVALID_TOKEN_ID) {
            if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
                throw std::runtime_error("tokenizer encountered unknown token")
    ↪ ;
            }
            ids.push_back(tokenizer.unk_id);
        } else {
            ids.push_back(token_id);
        }
        begin = end;
```

```
    }

    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.eos_id);
    }
    return ids;
}

std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
    std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
    hash = hash_string(hash, tokenizer.chat_template_hash);
    hash = hash_i32(hash, tokenizer.bos_id);
    hash = hash_i32(hash, tokenizer.eos_id);
    hash = hash_i32(hash, tokenizer.unk_id);
    for (const TokenEntry& entry : tokenizer.vocab) {
        hash = hash_string(hash, entry.text);
        hash = hash_i32(hash, entry.id);
    }
    return hash;
}

}  // namespace lcqi
```

四、命令行、账本、Trace 和 Benchmark

Listing 16.23 src/main.cpp

```
#include <lcqi/inference.hpp>
#include <lcqi/model.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;

std::vector<float> parse_input(int argc, char** argv, std::int32_t
    ↪ expected_size) {
    if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
        throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↩
    ↪  [repeat]");
```

```
    }
    std::vector<float> input(static_cast<std::size_t>(expected_size));
    for (std::int32_t i = 0; i < expected_size; ++i) {
        input[static_cast<std::size_t>(i)] =
            std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
    }
    return input;
}

std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
    const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
    if (argc <= repeat_index) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[repeat_index]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc < LCQI_EXPECTED_ARGC_MIN) {
            throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats> [repeat]");
        }

        const std::filesystem::path model_path = argv[1];
        const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
        const std::vector<float> input = parse_input(argc, argv, model.input_size);
        const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);

        const lcqi::InferenceResult result = lcqi::run_inference(model, input);
        std::cout << "predicted_class " << result.predicted_class << "\n";
        std::cout << "logits";
        for (const float value : result.logits) {
            std::cout << ' ' << value;
        }
        std::cout << "\n";

        const auto begin = std::chrono::steady_clock::now();
        std::int32_t checksum = 0;
        for (std::int32_t i = 0; i < repeat; ++i) {
            checksum += lcqi::run_inference(model, input).predicted_class;
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
```

```
        const double average_us =
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
        std::cout << "repeat " << repeat << "\n";
        std::cout << "average_us " << average_us << "\n";
        std::cout << "checksum " << checksum << "\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Listing 16.24 src/gpt2_cli.cpp

```
#include <lcqi/gpt2_reference.hpp>

#include <chrono>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace {

constexpr int LCQI_GPT2_TINY_ARGC = 1;
constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;

enum class Gpt2EngineMode {
    full_prefix,
    cached_kv,
};

struct Gpt2CliOptions {
    Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
    bool benchmark = false;
    std::filesystem::path model_dir;
    std::string prompt;
    std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
};

struct Gpt2BenchmarkTimings {
    double load_ms = 0.0;
```

```
    double tokenize_ms = 0.0;
    double prefill_ms = 0.0;
    double decode_ms = 0.0;
    double generate_ms = 0.0;
    double total_ms = 0.0;
    std::size_t prefill_steps = 0;
    std::size_t decode_steps = 0;
    std::size_t kv_cache_bytes = 0;
};

using Clock = std::chrono::steady_clock;

void print_ids(std::span<const std::int32_t> ids) {
    for (std::size_t index = 0; index < ids.size(); ++index) {
        if (index != 0) {
            std::cout << ' ';
        }
        std::cout << ids[index];
    }
}

[[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end)
    {
    return std::chrono::duration<double, std::milli>(end - start).count();
}

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

[[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
    switch (engine) {
        case Gpt2EngineMode::full_prefix:
            return "full_prefix";
        case Gpt2EngineMode::cached_kv:
            return "cached_kv";
    }
    return "unknown";
}

[[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
    if (value == "full" || value == "full_prefix") {
        return Gpt2EngineMode::full_prefix;
    }
    if (value == "cached" || value == "cached_kv") {
        return Gpt2EngineMode::cached_kv;
    }
    throw std::runtime_error("unknown --engine value, expected cached or full")
    ;
```

```
}

[[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
                                                  const char* name) {
    const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return value;
}

[[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
    Gpt2CliOptions options;
    std::vector<std::string> positional;
    for (int index = 1; index < argc; ++index) {
        const std::string arg = argv[index];
        if (arg == "--benchmark") {
            options.benchmark = true;
        } else if (arg == "--engine") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--engine requires a value");
            }
            ++index;
            options.engine = parse_engine(argv[index]);
        } else if (arg.rfind("--engine=", 0) == 0) {
            options.engine = parse_engine(std::string_view(arg).substr(std::
    ↪ string("--engine=").size()));
        } else if (arg == "--full") {
            options.engine = Gpt2EngineMode::full_prefix;
        } else if (arg == "--cached") {
            options.engine = Gpt2EngineMode::cached_kv;
        } else if (!arg.empty() && arg.front() == '-') {
            throw std::runtime_error("unknown option: " + arg);
        } else {
            positional.push_back(arg);
        }
    }

    if (positional.size() < 2 || positional.size() > 3) {
        throw std::runtime_error(
            "usage: lcqi_gpt2 [--engine cached|full] [--benchmark] "
            "model_dir prompt [max_new_tokens]");
    }
    options.model_dir = positional[0];
    options.prompt = positional[1];
    if (positional.size() == 3) {
        options.max_new_tokens = parse_non_negative_i32(positional[2], "
    ↪ max_new_tokens");
    }
    return options;
}

```

```cpp
[[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings) {
    const Clock::time_point generate_start = Clock::now();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);
        ++timings.decode_steps;
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    const Clock::time_point generate_end = Clock::now();
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    timings.decode_ms = timings.generate_ms;
    return tokens;
}

[[nodiscard]] std::vector<std::int32_t> generate_cached(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings) {
    if (prompt_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    lcqi::Gpt2KvCache cache(model.config);
    timings.kv_cache_bytes = cache.byte_size();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }

    const Clock::time_point generate_start = Clock::now();
    lcqi::Gpt2ForwardResult result;
    const Clock::time_point prefill_start = Clock::now();
    for (const std::int32_t token_id : prompt_ids) {
        result = lcqi::run_gpt2_forward_cached(model, cache, token_id);
        ++timings.prefill_steps;
    }
    const Clock::time_point prefill_end = Clock::now();
```

```
    timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);

    const Clock::time_point decode_start = Clock::now();
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            result = lcqi::run_gpt2_forward_cached(model, cache, result.predicted_token);
            ++timings.decode_steps;
        }
    }
    const Clock::time_point decode_end = Clock::now();
    const Clock::time_point generate_end = Clock::now();
    timings.decode_ms = elapsed_ms(decode_start, decode_end);
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    return tokens;
}

void print_benchmark(const Gpt2BenchmarkTimings& timings,
                     std::size_t prompt_tokens,
                     std::size_t generated_tokens) {
    const double new_tokens = static_cast<double>(generated_tokens);
    const double generated_tokens_per_second =
        timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms : 0.0;
    const double decode_tokens_per_second =
        timings.decode_ms > 0.0
            ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.decode_ms
            : 0.0;

    std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
    std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
    std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
    std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
    std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
    std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
    std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
    std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
    std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
    std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
    std::cout << "benchmark_generate_tokens_per_second "
              << generated_tokens_per_second << "\n";
```

```
231     std::cout << "benchmark_decode_tokens_per_second "
232               << decode_tokens_per_second << "\n";
233     std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
234 }
235
236 int run_tiny_mode() {
237     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↩
        ↪ ();
238     const std::vector<std::int32_t> prompt{
239         LCQI_GPT2_TINY_PROMPT_0,
240         LCQI_GPT2_TINY_PROMPT_1,
241         LCQI_GPT2_TINY_PROMPT_2,
242     };
243     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt↩
        ↪ );
244     const std::vector<std::int32_t> generated =
245         lcqi::gpt2_generate_greedy(model, prompt, 2);
246     std::cout << "mode tiny\n";
247     std::cout << "prompt_ids ";
248     print_ids(prompt);
249     std::cout << "\n";
250     std::cout << "predicted_token " << result.predicted_token << "\n";
251     std::cout << "logits";
252     for (const float value : result.logits) {
253         std::cout << ' ' << value;
254     }
255     std::cout << "\n";
256     std::cout << "generated_ids ";
257     print_ids(generated);
258     std::cout << "\n";
259     return EXIT_SUCCESS;
260 }
261
262 int run_real_model_mode(int argc, char** argv) {
263     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
264         throw std::runtime_error(
265             "usage: lcqi_gpt2 [--engine cached|full] [--benchmark] "
266             "model_dir prompt [max_new_tokens]");
267     }
268     const Clock::time_point total_start = Clock::now();
269     const Gpt2CliOptions options = parse_options(argc, argv);
270
271     Gpt2BenchmarkTimings timings;
272     const Clock::time_point load_start = Clock::now();
273     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(↩
        ↪ options.model_dir);
274     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
275         options.model_dir / "vocab.json",
276         options.model_dir / "merges.txt",
277         model.config.bos_token_id,
278         model.config.eos_token_id);
279     timings.load_ms = elapsed_ms(load_start, Clock::now());
```

```

    const Clock::time_point tokenize_start = Clock::now();
    const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, options.prompt);
    timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());

    std::vector<std::int32_t> generated;
    if (options.engine == Gpt2EngineMode::full_prefix) {
        generated = generate_full_prefix(model, prompt_ids, options.max_new_tokens, timings);
    } else {
        generated = generate_cached(model, prompt_ids, options.max_new_tokens, timings);
    }
    timings.total_ms = elapsed_ms(total_start, Clock::now());

    std::cout << "mode gpt2_directory\n";
    std::cout << "engine " << engine_name(options.engine) << "\n";
    std::cout << "model_dir " << options.model_dir.string() << "\n";
    std::cout << "prompt_ids ";
    print_ids(prompt_ids);
    std::cout << "\n";
    std::cout << "generated_ids ";
    print_ids(generated);
    std::cout << "\n";
    std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) << "\n";
    if (options.benchmark) {
        const std::size_t generated_tokens = generated.size() - prompt_ids.size();
        print_benchmark(timings, prompt_ids.size(), generated_tokens);
    }
    return EXIT_SUCCESS;
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc == LCQI_GPT2_TINY_ARGC) {
            return run_tiny_mode();
        }
        return run_real_model_mode(argc, argv);
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Listing 16.25 src/bench.cpp

```
#include <lcqi/int8_kernels.hpp>
```

```
#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;

const char* target_arch() noexcept {
#if defined(__x86_64__) || defined(_M_X64)
    return LCQI_TARGET_ARCH_X86_64;
#elif defined(__i386__) || defined(_M_IX86)
    return LCQI_TARGET_ARCH_I386;
#else
    return LCQI_TARGET_ARCH_UNKNOWN;
#endif
}

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[
    ↪ LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
    return {
        {128, 128, 16, repeat},
        {256, 256, 16, repeat},
        {512, 512, 16, repeat},
        {513, 257, 16, repeat},
        {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
                                  repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
        {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
         std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↩
    ↪ LCQI_LARGE_CASE_REPEAT_DIVISOR)},
```

```
    };
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::cout
            << "target_arch,input_size,output_size,output_block,repeat,backend,available,"
               "average_us,max_abs_diff,checksum\n";
        for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(repeat)) {
            const lcqi::KernelBenchmarkResult result =
                lcqi::benchmark_linear_i8_case(benchmark_case);
            for (const lcqi::KernelBackendBenchmark& backend : result.backends) {
                std::cout << target_arch() << ','
                          << result.benchmark_case.input_size << ','
                          << result.benchmark_case.output_size << ','
                          << result.benchmark_case.output_block_size << ','
                          << result.benchmark_case.repeat << ','
                          << backend.name << ','
                          << (backend.available ? 1 : 0) << ','
                          << backend.average_us << ','
                          << backend.max_abs_diff << ','
                          << backend.checksum << '\n';
            }
        }
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.26 src/trace.cpp

```
#include <lcqi/reference_decoder.hpp>
#include <lcqi/tokenizer.hpp>

#include <algorithm>
#include <array>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <sstream>
#include <string>
#include <string_view>
```

```cpp

namespace {

constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;

std::filesystem::path tokenizer_fixture_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& tokenizer,
                                                     std::string_view prompt) {
    std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, prompt);
    if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
        full_tokens.front() != LCQI_TRACE_BOS_ID ||
        full_tokens.back() != LCQI_TRACE_EOS_ID) {
        throw std::runtime_error("unexpected tokenizer fixture ids");
    }

    std::vector<std::int32_t> decoder_tokens(
        full_tokens.begin() + 1,
        full_tokens.end() - 1);
    if (!std::equal(decoder_tokens.begin(),
                    decoder_tokens.end(),
                    LCQI_TRACE_TOKEN_IDS.begin(),
                    LCQI_TRACE_TOKEN_IDS.end())) {
        throw std::runtime_error("unexpected decoder token ids");
    }
    return decoder_tokens;
}

std::string join_ints(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << 'x';
        }
        stream << values[index];
    }
    return stream.str();
}

std::string join_int_values(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
```

```
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    return stream.str();
}

std::int32_t checksum_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result += value;
    }
    return result;
}

std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result = std::max(result, std::abs(value));
    }
    return result;
}

std::string join_floats(std::span<const float> values) {
    std::ostringstream stream;
    const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
    for (std::size_t index = 0; index < count; ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
        stream << ";...";
    }
    return stream.str();
}

void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
                     const lcqi::TokenizerModel& tokenizer,
                     std::span<const std::int32_t> full_tokens) {
    std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↩
    ↪ checksum,max_abs,values\n";
    std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
    std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
              << ',' << full_tokens.size() << ",1,i32,contiguous,"
              << checksum_ints(full_tokens) << ','
              << max_abs_ints(full_tokens) << ','
              << join_int_values(full_tokens) << '\n';
    std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
```

```
                << ",scalar,scalar,u64,fnv1a,"
                << tokenizer_contract_hash(tokenizer) << ",0,"
                << tokenizer.chat_template_hash << '\n';
    for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)
    ↪ {
        std::cout << checkpoint.name << ','
                  << checkpoint.token_position << ','
                  << checkpoint.layer_id << ','
                  << join_ints(checkpoint.shape) << ','
                  << join_ints(checkpoint.stride) << ','
                  << checkpoint.dtype << ','
                  << checkpoint.layout << ','
                  << checkpoint.checksum << ','
                  << checkpoint.max_abs << ','
                  << join_floats(checkpoint.values) << '\n';
    }
    std::cout << "sampler_argmax,"
              << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
              << LCQI_TRACE_METADATA_LAYER
              << ',' << trace.result.logits.size() << ",1,f32,argmax,"
              << "0,0,token=" << trace.result.predicted_token << '\n';
    std::cout << "generated_token,"
              << LCQI_TRACE_TOKEN_IDS.size() << ','
              << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,i32,generated,"
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << '\n';
}

}  // namespace

int main() {
    try {
        const lcqi::TokenizerModel tokenizer =
            lcqi::load_tokenizer(tokenizer_fixture_path());
        const std::vector<std::int32_t> full_tokens =
            lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const std::vector<std::int32_t> decoder_tokens =
            decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const lcqi::ReferenceDecoderModel model = lcqi::
    ↪ make_tiny_reference_decoder_model();
        const lcqi::ReferenceDecodeTraceResult trace =
            lcqi::run_reference_decode_with_trace(model, decoder_tokens);
        print_trace_csv(trace, tokenizer, full_tokens);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.27 src/ledger.cpp

```cpp
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <iomanip>
#include <iostream>
#include <string_view>

namespace {

constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr double LCQI_FLOPS_PER_MAC = 2.0;
constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;

struct DTypeLedger {
    std::string_view name;
    double weight_gb_per_token = 0.0;
    double kv_element_bytes = 0.0;
};

constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
    {"fp16", 14.0, 2.0},
    {"int8", 7.0, 2.0},
    {"int4", 3.7, 2.0},
};

constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
    50.0,
    100.0,
    200.0,
    600.0,
    1000.0,
    1600.0,
};

double linear_macs_per_layer() noexcept {
    const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
    const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
    const double qkv = hidden * (hidden + 2.0 * kv_width);
    const double output_projection = hidden * hidden;
    const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
    return qkv + output_projection + mlp;
}

double attention_macs_per_layer() noexcept {
```

```cpp
    return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
           LCQI_SEVENB_HEAD_DIM;
}

double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
    return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
}

double kv_read_gb_per_decode_token(double element_bytes) noexcept {
    const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_GB;
}

double kv_capacity_mib(double element_bytes, std::int32_t context_length) noexcept {
    const double bytes = static_cast<double>(context_length) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_MIB;
}

void print_model_rows() {
    const double linear_macs = linear_macs_per_layer();
    const double attention_macs = attention_macs_per_layer();
    const double total_flops =
        (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * LCQI_FLOPS_PER_MAC;
    std::cout << "section,item,value,unit,notes\n";
    std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B fixture\n";
    std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
    std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA query heads\n";
    std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV heads\n";
    std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per head\n";
    std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,decode token\n";
    std::cout << "compute,attention_macs_per_layer," << attention_macs
              << ",macs,context=4096\n";
    std::cout << "compute,total_decode_flops_per_token," << total_flops
              << ",flops,linear plus attention\n";
}

void print_kv_rows() {
    for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
        std::cout << "kv,fp16_capacity_context_" << context << ','
                  << kv_capacity_mib(2.0, context)
                  << ",MiB,single session\n";
    }
```

```
}

void print_dtype_rows() {
    for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
        const double kv_read_gb = kv_read_gb_per_decode_token(dtype.
    ↪ kv_element_bytes);
        const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
        std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
                  << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
        std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
                  << kv_read_gb << ",GB/token,context=4096\n";
        std::cout << "decode_bytes," << dtype.name << "_total_gb,"
                  << total_gb << ",GB/token,weight plus KV\n";
        for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
            std::cout << "bandwidth_limit," << dtype.name << "_at_"
                      << static_cast<std::int32_t>(bandwidth) << "_gbps,"
                      << bandwidth / total_gb
                      << ",tokens/s,bandwidth-only upper bound\n";
        }
    }
}

}  // namespace

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        print_model_rows();
        print_kv_rows();
        print_dtype_rows();
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.28 src/serving_probe.cpp

```
#include <algorithm>
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string_view>
#include <vector>

namespace {

constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
```

```
constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;

struct ProbeRequest {
    std::string_view request_id;
    std::int32_t prompt_tokens = 0;
    std::int32_t max_new_tokens = 0;
    bool cancel_after_prefill = false;
};

struct ProbeResult {
    std::string_view request_id;
    std::int32_t accepted = 0;
    std::string_view rejection_reason;
    std::int32_t prompt_tokens = 0;
    std::int32_t reserved_tokens = 0;
    double kv_reserved_mib = 0.0;
    double queue_wait_ms = 0.0;
    double prefill_ms = 0.0;
    double ttft_ms = 0.0;
    double decode_step_p95_ms = 0.0;
    double tpot_ms = 0.0;
    std::int32_t completion_tokens = 0;
    std::string_view finish_reason;
    double cancel_reclaim_ms = 0.0;
};

const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
    {"req_short", 32, 4, false},
    {"req_rag", 512, 8, false},
    {"req_cancel", 128, 16, true},
    {"req_too_long", 4096, 512, false},
};

double kv_mib_for_tokens(std::int32_t tokens) noexcept {
    const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *↩
    ↪  2.0 *
                         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
                         LCQI_PROBE_KV_ELEMENT_BYTES;
    return bytes / LCQI_BYTES_PER_MIB;
}
```

```cpp

double percentile(std::vector<double> values, double fraction) {
    if (values.empty()) {
        return 0.0;
    }
    std::sort(values.begin(), values.end());
    const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
    const std::size_t index =
        std::min(max_index,
                 static_cast<std::size_t>(
                     std::ceil(fraction * static_cast<double>(max_index))));
    return values[index];
}

std::vector<ProbeResult> run_probe() {
    std::vector<ProbeResult> results;
    double reserved_kv_mib = 0.0;
    std::int32_t accepted_before = 0;
    for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
        ProbeResult result;
        result.request_id = request.request_id;
        result.prompt_tokens = request.prompt_tokens;
        result.reserved_tokens = request.prompt_tokens + request.max_new_tokens;
        result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
        const double static_session_mib =
            result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
            LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
        if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
            result.accepted = 0;
            result.rejection_reason = "memory_budget_exceeded";
            result.finish_reason = "rejected";
            results.push_back(result);
            continue;
        }

        reserved_kv_mib += result.kv_reserved_mib;
        result.accepted = 1;
        result.rejection_reason = "none";
        result.queue_wait_ms = static_cast<double>(accepted_before) *
                               LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
        result.prefill_ms = static_cast<double>(request.prompt_tokens) *
                            LCQI_PROBE_PREFILL_MS_PER_TOKEN;
        result.ttft_ms = result.queue_wait_ms + result.prefill_ms +
    LCQI_PROBE_DECODE_STEP_MS;
        result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
                                    static_session_mib / 512.0;
        result.tpot_ms = result.decode_step_p95_ms;
        if (request.cancel_after_prefill) {
            result.completion_tokens = 0;
            result.finish_reason = "cancelled";
```

```
            result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
            reserved_kv_mib -= result.kv_reserved_mib;
        } else {
            result.completion_tokens = request.max_new_tokens;
            result.finish_reason = "max_tokens";
        }
        ++accepted_before;
        results.push_back(result);
    }
    return results;
}

void print_request_rows(const std::vector<ProbeResult>& results) {
    std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,"
                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,"
                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,"
                 "metric_value\n";
    for (const ProbeResult& result : results) {
        std::cout << "request," << result.request_id << ','
                  << result.accepted << ','
                  << result.rejection_reason << ','
                  << result.prompt_tokens << ','
                  << result.reserved_tokens << ','
                  << result.kv_reserved_mib << ','
                  << result.queue_wait_ms << ','
                  << result.prefill_ms << ','
                  << result.ttft_ms << ','
                  << result.decode_step_p95_ms << ','
                  << result.tpot_ms << ','
                  << result.completion_tokens << ','
                  << result.finish_reason << ','
                  << result.cancel_reclaim_ms << ",,\n";
    }
}

void print_summary_row(const std::vector<ProbeResult>& results) {
    std::vector<double> ttft_values;
    std::vector<double> queue_values;
    double accepted = 0.0;
    double rejected = 0.0;
    double cancelled = 0.0;
    double reserved_peak_mib = 0.0;
    double running_reserved_mib = 0.0;
    for (const ProbeResult& result : results) {
        if (result.accepted == 0) {
            rejected += 1.0;
            continue;
        }
        accepted += 1.0;
```

```
        ttft_values.push_back(result.ttft_ms);
        queue_values.push_back(result.queue_wait_ms);
        running_reserved_mib += result.kv_reserved_mib;
        reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
        if (result.finish_reason == "cancelled") {
            cancelled += 1.0;
            running_reserved_mib -= result.kv_reserved_mib;
        }
    }
    const double total = accepted + rejected;
    const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "accepted_count," << accepted << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
              << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejected_count," << rejected << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejection_rate," << rejection_rate << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
}

}  // namespace

int main() {
    try {
        std::cout << std::fixed << std::setprecision(3);
        const std::vector<ProbeResult> results = run_probe();
        print_request_rows(results);
        print_summary_row(results);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

Listing 16.29 src/onednn_bench.cpp

```
#include <dnnl.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iostream>
```

```cpp
#include <numeric>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <vector>

namespace {

constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
constexpr std::int64_t LCQI_ONEDNN_K = 4096;
constexpr std::int64_t LCQI_ONEDNN_O = 4096;
constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr float LCQI_INPUT_SCALE = 0.125F;
constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_CENTER = 8;
constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
constexpr float LCQI_WEIGHT_SCALE = 0.03125F;

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_ONEDNN_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[
    ↪ LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<float> make_input() {
    std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
    for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
        input[static_cast<std::size_t>(index)] =
            static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER
    ↪ ) *
            LCQI_INPUT_SCALE;
    }
    return input;
}

std::vector<float> make_weights_kn() {
    std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * 
    ↪ LCQI_ONEDNN_O));
    for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
        for (std::int64_t out = 0; out < LCQI_ONEDNN_O; ++out) {
            const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
            const float value =
```

```
                static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
                                   LCQI_WEIGHT_CENTER) *
                LCQI_WEIGHT_SCALE;
            weights[static_cast<std::size_t>(k * LCQI_ONEDNN_O + out)] = value;
        }
    }
    return weights;
}

float checksum(const std::vector<float>& values) {
    return std::accumulate(values.begin(), values.end(), 0.0F);
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::vector<float> input = make_input();
        std::vector<float> weights = make_weights_kn();
        std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * LCQI_ONEDNN_O),
                                  0.0F);

        dnnl::engine engine(dnnl::engine::kind::cpu, 0);
        dnnl::stream stream(engine);

        const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_K};
        const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_O};
        const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_O};

        const dnnl::memory::desc source_desc(
            source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
        const dnnl::memory::desc weight_desc(
            weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
        const dnnl::memory::desc output_desc(
            output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);

        dnnl::memory source_memory(source_desc, engine, input.data());
        dnnl::memory weight_memory(weight_desc, engine, weights.data());
        dnnl::memory output_memory(output_desc, engine, output.data());

        dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, weight_desc, output_desc);
        dnnl::matmul matmul(matmul_desc);
        const std::unordered_map<int, dnnl::memory> arguments = {
            {DNNL_ARG_SRC, source_memory},
```

```
            {DNNL_ARG_WEIGHTS, weight_memory},
            {DNNL_ARG_DST, output_memory},
        };

        for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
            matmul.execute(stream, arguments);
            stream.wait();
        }

        const auto begin = std::chrono::steady_clock::now();
        for (std::int32_t index = 0; index < repeat; ++index) {
            matmul.execute(stream, arguments);
            stream.wait();
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
        const double average_us =
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);

        std::cout << "library,input_size,output_size,dtype,repeat,average_us,checksum\n";
        std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_O
                  << ",f32," << repeat << ',' << average_us << ','
                  << checksum(output) << '\n';
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

五、测试和模型 Fixture

Listing 16.30 tests/lcqi_tests.cpp

```
#include <lcqi/gpt2_reference.hpp>
#include <lcqi/inference.hpp>
#include <lcqi/int8_kernels.hpp>
#include <lcqi/model.hpp>
#include <lcqi/reference_decoder.hpp>
#include <lcqi/safetensors.hpp>
#include <lcqi/tokenizer.hpp>

#include <array>
#include <cmath>
#include <cstring>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iostream>
```

```cpp
#include <span>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

namespace {

constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↩
    ↪ LCQI_REFERENCE_DECODER_LOGITS{
    0.352516979F,
    -0.126884326F,
    0.0797837451F,
    -0.29797405F,
    0.127030417F,
};
constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 1345290284538833373 4ULL;
constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
    0.407358F,
    -0.504049F,
```

```
    -0.107834F,
    0.840337F,
    -0.450123F,
    -0.156763F,
};
constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;

struct KernelShapeCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
};

struct RawTensorFixture {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::vector<std::uint8_t> bytes;
};

struct FloatTensorFixture {
    std::string name;
    std::vector<std::int64_t> shape;
    std::vector<float> values;
};

constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
    {17, 19, 8},
    {33, 7, 8},
    {64, 32, 16},
    {65, 31, 16},
}};

void require(bool condition, const char* message) {
    if (!condition) {
        throw std::runtime_error(message);
    }
}

void require_close(float actual, float expected, const char* message) {
    if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
        throw std::runtime_error(message);
    }
}

std::filesystem::path test_model_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
```

```
            "models" / "tiny_mlp_i8.txt";
}

std::filesystem::path test_tokenizer_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
            "models" / "tiny_tokenizer.txt";
}

std::filesystem::path temp_file_path(const std::string& name) {
    return std::filesystem::temp_directory_path() / name;
}

void write_le_u64(std::ofstream& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        const char byte = static_cast<char>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
        output.write(&byte, 1);
    }
}

void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
    output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
    output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
    ↪  LCQI_BYTE_MASK));
}

void append_le_f32(std::vector<std::uint8_t>& output, float value) {
    std::uint32_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    for (std::int32_t i = 0; i < 4; ++i) {
        output.push_back(static_cast<std::uint8_t>(
            (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void write_safetensors_fixture(const std::filesystem::path& path,
                               const std::string& header,
                               std::uint64_t payload_bytes) {
    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors test fixture");
    }
    write_le_u64(output, static_cast<std::uint64_t>(header.size()));
    output.write(header.data(), static_cast<std::streamsize>(header.size()));
    for (std::uint64_t i = 0; i < payload_bytes; ++i) {
        const char byte = static_cast<char>(i & 0xFFU);
        output.write(&byte, 1);
    }
}

std::string json_shape(std::span<const std::int64_t> shape) {
    std::ostringstream stream;
```

```
    stream << '[';
    for (std::size_t index = 0; index < shape.size(); ++index) {
        if (index != 0) {
            stream << ',';
        }
        stream << shape[index];
    }
    stream << ']';
    return stream.str();
}

void write_safetensors_raw_fixture(const std::filesystem::path& path,
                                   std::span<const RawTensorFixture> tensors) {
    std::ostringstream header;
    header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
    std::uint64_t offset = 0;
    for (const RawTensorFixture& tensor : tensors) {
        header << ",\"" << tensor.name << "\":{\"dtype\":\"" << tensor.dtype
               << "\",\"shape\":" << json_shape(tensor.shape)
               << ",\"data_offsets\":[" << offset << ','
               << offset + tensor.bytes.size() << "]}";
        offset += tensor.bytes.size();
    }
    header << '}';

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors raw test fixture");
    }
    const std::string header_text = header.str();
    write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
    output.write(header_text.data(), static_cast<std::streamsize>(header_text.size()));
    for (const RawTensorFixture& tensor : tensors) {
        output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
                     static_cast<std::streamsize>(tensor.bytes.size()));
    }
}

std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
    std::vector<std::uint8_t> bytes;
    bytes.reserve(values.size() * 4);
    for (const float value : values) {
        append_le_f32(bytes, value);
    }
    return bytes;
}

std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& linear) {
    std::vector<float> transposed(
        static_cast<std::size_t>(linear.input_size) *
```

```
            static_cast<std::size_t>(linear.output_size),
        0.0F);
    for (std::int32_t out = 0; out < linear.output_size; ++out) {
        for (std::int32_t in = 0; in < linear.input_size; ++in) {
            transposed[static_cast<std::size_t>(in) *
                           static_cast<std::size_t>(linear.output_size) +
                       static_cast<std::size_t>(out)] =
                linear.weights[static_cast<std::size_t>(out) *
                                   static_cast<std::size_t>(linear.input_size) +
                               static_cast<std::size_t>(in)];
        }
    }
    return transposed;
}

void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
                    std::string name,
                    std::vector<std::int64_t> shape,
                    std::span<const float> values) {
    tensors.push_back(RawTensorFixture{
        std::move(name),
        "F32",
        std::move(shape),
        f32_payload(values),
    });
}

void write_text_file(const std::filesystem::path& path, const std::string& text) {
    std::ofstream output(path);
    if (!output) {
        throw std::runtime_error("cannot create text fixture");
    }
    output << text;
}

void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
                                 const lcqi::Gpt2ReferenceModel& model) {
    std::vector<RawTensorFixture> tensors;
    add_f32_tensor(tensors,
                   "transformer.wte.weight",
                   {model.config.vocab_size, model.config.hidden_size},
                   model.token_embedding);
    add_f32_tensor(tensors,
                   "transformer.wpe.weight",
                   {model.config.max_positions, model.config.hidden_size},
                   model.position_embedding);
    for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++layer_id) {
        const lcqi::Gpt2LayerWeightsF32& layer =
            model.layers[static_cast<std::size_t>(layer_id)];
```

```
269         const std::string prefix = "transformer.h." + std::to_string(layer_id) ↩
    ↪ + ".";
270         add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.↩
    ↪ hidden_size}, layer.ln_1_weight);
271         add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size↩
    ↪ }, layer.ln_1_bias);
272         const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(↩
    ↪ layer.c_attn);
273         add_f32_tensor(tensors,
274                        prefix + "attn.c_attn.weight",
275                        {model.config.hidden_size,
276                         model.config.hidden_size * 3},
277                        c_attn_hf);
278         add_f32_tensor(tensors,
279                        prefix + "attn.c_attn.bias",
280                        {model.config.hidden_size * 3},
281                        layer.c_attn.bias);
282         const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(↩
    ↪ layer.c_proj);
283         add_f32_tensor(tensors,
284                        prefix + "attn.c_proj.weight",
285                        {model.config.hidden_size, model.config.hidden_size},
286                        c_proj_hf);
287         add_f32_tensor(tensors,
288                        prefix + "attn.c_proj.bias",
289                        {model.config.hidden_size},
290                        layer.c_proj.bias);
291         add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.↩
    ↪ hidden_size}, layer.ln_2_weight);
292         add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size↩
    ↪ }, layer.ln_2_bias);
293         const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.↩
    ↪ c_fc);
294         add_f32_tensor(tensors,
295                        prefix + "mlp.c_fc.weight",
296                        {model.config.hidden_size, model.config.↩
    ↪ intermediate_size},
297                        c_fc_hf);
298         add_f32_tensor(tensors,
299                        prefix + "mlp.c_fc.bias",
300                        {model.config.intermediate_size},
301                        layer.c_fc.bias);
302         const std::vector<float> mlp_proj_hf =
303             transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
304         add_f32_tensor(tensors,
305                        prefix + "mlp.c_proj.weight",
306                        {model.config.intermediate_size, model.config.↩
    ↪ hidden_size},
307                        mlp_proj_hf);
308         add_f32_tensor(tensors,
309                        prefix + "mlp.c_proj.bias",
310                        {model.config.hidden_size},
```

```
                        layer.mlp_c_proj.bias);
    }
    add_f32_tensor(tensors,
                   "transformer.ln_f.weight",
                   {model.config.hidden_size},
                   model.final_ln_weight);
    add_f32_tensor(tensors,
                   "transformer.ln_f.bias",
                   {model.config.hidden_size},
                   model.final_ln_bias);
    write_safetensors_raw_fixture(path, tensors);
}

void test_tiny_mlp() {
    const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
    require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
    require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch");
    require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch");

    const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
    const lcqi::InferenceResult result = lcqi::run_inference(model, input);

    require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected predicted class");
    require(result.logits.size() == static_cast<std::size_t>(LCQI_TEST_OUTPUT_SIZE),
            "unexpected logits size");
    require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
    require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");

    std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), 0.0F);
    lcqi::linear_i8(model.hidden, input, hidden);
    lcqi::relu_inplace(hidden);
    require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
    require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
    require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
}

void test_tokenizer_contract() {
    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(test_tokenizer_path());
    require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
    require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
    require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
    require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
            "tokenizer template hash mismatch");

    const std::vector<std::int32_t> token_ids =
        lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
```

```
    const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
    require(token_ids == expected, "tokenizer prompt ids mismatch");
    require(lcqi::tokenizer_contract_hash(tokenizer) == LCQI_TOKENIZER_HASH_EXPECTED,
            "tokenizer contract hash mismatch");
}

void test_tokenizer_rejects_unknown_without_unk() {
    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(test_tokenizer_path());
    bool threw = false;
    try {
        static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    require(threw, "tokenizer should reject unknown token when unk id is absent");
}

void test_safetensors_manifest_contract() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
    const std::string header =
        R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight":{"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets":[24,48]}})";
    write_safetensors_fixture(path, header, LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);

    const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(path);
    std::filesystem::remove(path);

    require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
            "safetensors header size mismatch");
    require(manifest.data_start_offset ==
                static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
                    static_cast<std::uint64_t>(header.size()),
            "safetensors data start mismatch");
    require(manifest.metadata.size() == 1, "safetensors metadata count mismatch");
    require(manifest.metadata[0].first == "format" &&
                manifest.metadata[0].second == "lcqi-fixture",
            "safetensors metadata mismatch");
    require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");

    const lcqi::SafeTensorEntry* embedding =
        manifest.find_tensor("model.embed_tokens.weight");
    require(embedding != nullptr, "safetensors embedding tensor missing");
```

```
399     require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
400     require(embedding->shape == std::vector<std::int64_t>({2, 3}),
401             "safetensors embedding shape mismatch");
402     require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
403                 embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
404             "safetensors embedding offset mismatch");
405     require(embedding->byte_size() ==
406                 LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
407             "safetensors embedding byte size mismatch");
408
409     const lcqi::SafeTensorEntry* q_proj =
410         manifest.find_tensor("model.layers.0.attn.q_proj.weight");
411     require(q_proj != nullptr, "safetensors q projection tensor missing");
412     require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
413     require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
414                 q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
415             "safetensors q projection offset mismatch");
416 }
417
418 void test_safetensors_rejects_bad_offsets() {
419     const std::filesystem::path path =
420         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
421     const std::string header =
422         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}})";
423     write_safetensors_fixture(path, header, 4);
424
425     bool threw = false;
426     try {
427         static_cast<void>(lcqi::load_safetensors_manifest(path));
428     } catch (const std::runtime_error&) {
429         threw = true;
430     }
431     std::filesystem::remove(path);
432     require(threw, "safetensors parser should reject offsets beyond payload");
433 }
434
435 void test_safetensors_rejects_bad_dtype_shape_size() {
436     const std::filesystem::path path =
437         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
438     const std::string header =
439         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}})";
440     write_safetensors_fixture(path, header, 8);
441
442     bool threw = false;
443     try {
444         static_cast<void>(lcqi::load_safetensors_manifest(path));
445     } catch (const std::runtime_error&) {
446         threw = true;
447     }
448     std::filesystem::remove(path);
449     require(threw, "safetensors parser should reject dtype/shape byte mismatch"↩
    ↪ );
```

```
}

void test_safetensors_reads_float_tensors() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_float_read.safetensors");
    std::vector<std::uint8_t> f16_bytes;
    append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
    append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
    std::vector<std::uint8_t> bf16_bytes;
    append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
    append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);

    const std::array<RawTensorFixture, 3> tensors{{
        {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
        {"f16", "F16", {2}, f16_bytes},
        {"bf16", "BF16", {2}, bf16_bytes},
    }};
    write_safetensors_raw_fixture(path, tensors);

    const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
    std::filesystem::remove(path);
    const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32");
    const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16");
    const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "bf16");

    require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
    require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
    require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
    require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
    require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
    require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
}

void test_gpt2_tiny_forward_and_generation() {
    const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model();
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);

    require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size mismatch");
    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
            "GPT-2 predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
        require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 logit mismatch");
    }

```

```
    const std::vector<std::int32_t> generated =
        lcqi::gpt2_generate_greedy(model, tokens, 2);
    const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
    require(generated == expected, "GPT-2 greedy generation mismatch");
    require(static_cast<std::int32_t>(generated.size()) == LCQI_GPT2_GENERATED_LENGTH,
            "GPT-2 generated length mismatch");

    lcqi::Gpt2KvCache cache(model.config);
    lcqi::Gpt2ForwardResult cached_result;
    for (const std::int32_t token : tokens) {
        cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
    }
    require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
            "GPT-2 KV cache filled token count mismatch");
    require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero");
    require(cached_result.predicted_token == result.predicted_token,
            "cached GPT-2 predicted token mismatch");
    require(cached_result.logits.size() == result.logits.size(),
            "cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(cached_result.logits[index],
                      result.logits[index],
                      "cached GPT-2 logit mismatch");
    }

    const std::vector<std::int32_t> cached_generated =
        lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
    require(cached_generated == expected, "cached GPT-2 greedy generation mismatch");
}

void test_gpt2_byte_bpe_tokenizer() {
    const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.json");
    const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.txt");
    const std::string space_marker = "\xC4\xA0";
    const std::string vocab =
        "{\"Hello\":7,\",\":8,\"" + space_marker + "world\":9,\"!\":10}";
    const std::string merges =
        "#version: 0.2\n"
        "H e\n"
        "He l\n"
        "Hel l\n"
        "Hell o\n" +
        space_marker + " w\n" +
        space_marker + "w o\n" +
        space_marker + "wo r\n" +
        space_marker + "wor l\n" +
        space_marker + "worl d\n";
```

```
    write_text_file(vocab_path, vocab);
    write_text_file(merges_path, merges);

    const lcqi::Gpt2Tokenizer tokenizer =
        lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
    std::filesystem::remove(vocab_path);
    std::filesystem::remove(merges_path);

    const std::vector<std::int32_t> ids =
        lcqi::gpt2_encode(tokenizer, "Hello, world!");
    const std::vector<std::int32_t> expected{
        LCQI_GPT2_TOKENIZER_HELLO_ID,
        8,
        9,
        10,
    };
    require(ids == expected, "GPT-2 BPE ids mismatch");
    require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
            "GPT-2 BPE decode mismatch");
}

void test_gpt2_loads_hf_style_tiny_checkpoint() {
    const std::filesystem::path directory =
        temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
    std::filesystem::remove_all(directory);
    std::filesystem::create_directories(directory);

    const lcqi::Gpt2ReferenceModel original = lcqi::make_tiny_gpt2_reference_model();
    write_text_file(
        directory / "config.json",
        R"({"model_type":"gpt2","n_embd":4,"n_head":2,"n_layer":1,"n_positions":8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
    write_tiny_gpt2_safetensors(directory / "model.safetensors", original);

    const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(directory);
    const std::vector<std::int32_t> tokens{1, 2, 3};
    const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, tokens);
    std::filesystem::remove_all(directory);

    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
            "loaded GPT-2 predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
        require_close(result.logits[index],
                      LCQI_GPT2_LOGITS[index],
                      "loaded GPT-2 logit mismatch");
    }
}

```

```cpp
void test_gpt2_loader_rejects_bad_shape() {
    const std::filesystem::path directory =
        temp_file_path("lcqi_gpt2_bad_shape_dir");
    std::filesystem::remove_all(directory);
    std::filesystem::create_directories(directory);

    const lcqi::Gpt2ReferenceModel original = lcqi::make_tiny_gpt2_reference_model();
    write_text_file(
        directory / "config.json",
        R"({"model_type":"gpt2","n_embd":5,"n_head":1,"n_layer":1,"n_positions":8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
    write_tiny_gpt2_safetensors(directory / "model.safetensors", original);

    bool threw = false;
    try {
        static_cast<void>(lcqi::load_gpt2_from_directory(directory));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    std::filesystem::remove_all(directory);
    require(threw, "GPT-2 loader should reject bad tensor shape");
}

void test_reference_rms_norm() {
    const std::vector<float> input{3.0F, 4.0F};
    const std::vector<float> weight{1.0F, 0.5F};
    std::vector<float> output(2, 0.0F);
    lcqi::rms_norm(input, weight, 0.0F, output);
    require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
    require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
}

void test_reference_rope() {
    std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
    lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
    require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
    require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
    require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine mismatch");
    require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
}

void test_reference_kv_attention() {
    lcqi::DecoderConfig config;
    config.hidden_size = 4;
    config.query_heads = 2;
    config.kv_heads = 1;
    config.head_dim = 2;
    config.intermediate_size = 4;
    config.vocab_size = 4;
```

```
637     config.max_context = 4;
638
639     lcqi::ReferenceKVCache cache(config, 1);
640     const std::vector<float> key{1.0F, 0.0F};
641     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ↩
        ↪ LCQI_ATTENTION_EXPECTED_1};
642     cache.append(0, 0, key, value);
643     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
644     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
645
646     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
647     std::vector<float> output(4, 0.0F);
648     lcqi::attention_decode(config, cache, 0, 0, query, output);
649     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0↩
        ↪  mismatch");
650     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1↩
        ↪  mismatch");
651     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0↩
        ↪  mismatch");
652     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1↩
        ↪  mismatch");
653 }
654
655 void test_reference_decoder_end_to_end() {
656     const lcqi::ReferenceDecoderModel model = lcqi::↩
        ↪ make_tiny_reference_decoder_model();
657     const std::vector<std::int32_t> tokens{1, 2, 3};
658     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model↩
        ↪ , tokens);
659     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
660             "reference decoder logits size mismatch");
661     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
662             "reference decoder predicted token mismatch");
663     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ↩
        ↪ ++index) {
664         require_close(result.logits[index],
665                       LCQI_REFERENCE_DECODER_LOGITS[index],
666                       "reference decoder golden logit mismatch");
667     }
668 }
669
670 void test_reference_decoder_trace_contract() {
671     const lcqi::ReferenceDecoderModel model = lcqi::↩
        ↪ make_tiny_reference_decoder_model();
672     const std::vector<std::int32_t> tokens{1, 2, 3};
673     const lcqi::ReferenceDecodeTraceResult trace =
674         lcqi::run_reference_decode_with_trace(model, tokens);
675
676     require(trace.result.predicted_token == ↩
        ↪ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
677             "reference trace predicted token mismatch");
678     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
```

```
679
680     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
681     require(first.name == "embedding", "first checkpoint should be embedding");
682     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
683             "first checkpoint token position mismatch");
684     require(first.dtype == "f32", "first checkpoint dtype mismatch");
685     require(first.layout == "row_major", "first checkpoint layout mismatch");
686     require(first.shape.size() == 1 &&
687                 first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
688             "first checkpoint shape mismatch");
689     require(first.stride.size() == 1 && first.stride[0] == 1,
690             "first checkpoint stride mismatch");
691
692     bool saw_logits = false;
693     bool saw_kv_slot = false;
694     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↩
    ↪  {
695         if (checkpoint.name == "kv_cache_key_slot") {
696             saw_kv_slot = true;
697             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch"↩
    ↪ );
698             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout↩
    ↪  mismatch");
699         }
700         if (checkpoint.name == "logits") {
701             saw_logits = true;
702             require(checkpoint.token_position == ↩
    ↪ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
703                     "logits checkpoint token position mismatch");
704             require(checkpoint.shape.size() == 1 &&
705                         checkpoint.shape[0] ==
706                             static_cast<std::int32_t>(↩
    ↪ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
707                     "logits checkpoint shape mismatch");
708             require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↩
    ↪ size(),
709                     "logits checkpoint value size mismatch");
710             for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↩
    ↪ size(); ++index) {
711                 require_close(checkpoint.values[index],
712                               LCQI_REFERENCE_DECODER_LOGITS[index],
713                               "trace logits checkpoint mismatch");
714             }
715         }
716     }
717     require(saw_kv_slot, "KV checkpoint missing");
718     require(saw_logits, "logits checkpoint missing");
719 }
720
721 void test_packed_i8_kernel_matches_scalar() {
722     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
723         const lcqi::QuantizedLinearLayer layer =
```

```
724             lcqi::make_deterministic_i8_layer(shape_case.input_size,
725                                               shape_case.output_size,
726                                               0.03125F);
727         const lcqi::PackedLinearI8 packed =
728             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
729         const std::vector<float> input = lcqi::make_deterministic_input(layer.↩
    ↪ input_size);
730         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
731         std::vector<float> packed_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
732         lcqi::linear_i8(layer, input, scalar_output);
733         lcqi::linear_i8_packed(packed, input, packed_output);
734         require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↩
    ↪ LCQI_KERNEL_MAX_DIFF,
735                 "packed int8 kernel output differs from scalar");
736 
737         const lcqi::PackedLinearI8 simd_packed =
738             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
739         if (lcqi::linear_i8_packed_avx2_available()) {
740             std::vector<float> avx2_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
741             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
742             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↩
    ↪ LCQI_KERNEL_MAX_DIFF,
743                     "AVX2 int8 kernel output differs from scalar");
744         }
745     }
746 }
747 
748 }  // namespace
749 
750 int main() {
751     try {
752         test_tiny_mlp();
753         test_tokenizer_contract();
754         test_tokenizer_rejects_unknown_without_unk();
755         test_safetensors_manifest_contract();
756         test_safetensors_rejects_bad_offsets();
757         test_safetensors_rejects_bad_dtype_shape_size();
758         test_safetensors_reads_float_tensors();
759         test_gpt2_tiny_forward_and_generation();
760         test_gpt2_byte_bpe_tokenizer();
761         test_gpt2_loads_hf_style_tiny_checkpoint();
762         test_gpt2_loader_rejects_bad_shape();
763         test_reference_rms_norm();
764         test_reference_rope();
765         test_reference_kv_attention();
766         test_reference_decoder_end_to_end();
767         test_reference_decoder_trace_contract();
768         test_packed_i8_kernel_matches_scalar();
769 
```

```
        std::cout << "lcqi tests passed\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "lcqi test failure: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Listing 16.31 models/tiny_mlp_i8.txt

```
LCQI_MODEL_V1
input_size 4
hidden_size 3
output_size 2
hidden_scale 0.1
hidden_weights
5 -3 2 1
-4 6 1 -2
3 2 -5 4
hidden_bias
0.1 -0.2 0.0
output_scale 0.05
output_weights
4 -6 2
-3 5 8
output_bias
-0.5 0.4
```

Listing 16.32 models/tiny_tokenizer.txt

```
LCQI_TOKENIZER_V1
bos_id 0
eos_id 4
unk_id -1
chat_template_hash tiny-chat-template-v1
vocab_size 5
vocab
0 <bos>
1 tok1
2 tok2
3 tok3
4 <eos>
```

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。